ARCHITECTURE & IMPLEMENTATION SPECIFICATION

Account Intelligence SaaS Platform

Multi-tenant, agentic account research with a private-context vault

VERSION	2.0 (enhanced and corrected edition of v1)
DATE	1 October 2026
STATUS	Draft for engineering review
CLASSIFICATION	Confidential

Contents

Contents . . . 2

1. Executive Summary . . . 5

1.1 Ten decisions at a glance . . . 5

1.2 What changed from v1 (defects found and resolved) . . . 5

2. System Requirements . . . 7

2.1 Product scope . . . 7

2.2 Functional requirements . . . 7

2.3 Non-functional requirements (measurable) . . . 7

2.4 Capacity model . . . 8

3. Architecture Overview . . . 9

3.1 Design principles . . . 9

3.2 System context . . . 9

3.3 Container view . . . 9

3.4 Component responsibilities . . . 10

3.5 Architecture decision records (summary) . . . 11

4. Tenant Isolation Architecture . . . 12

4.1 Isolation tiers . . . 12

4.2 RLS correctness checklist . . . 12

4.3 Checkpoint and vector isolation . . . 13

4.4 Cryptographic isolation and offboarding . . . 13

4.5 Attack scenarios the design must survive . . . 13

5. Run Lifecycle and Durability . . . 14

5.1 Run state machine . . . 14

5.2 End-to-end sequence . . . 14

5.3 One owner per failure class . . . 15

5.4 Time-sliced execution . . . 16

5.5 Tool-call ledger: exactly-once billing . . . 16

5.6 Budgets and circuit breakers . . . 16

5.7 Version safety . . . 16

6. Research Agent Design . . . 17

6.1 Nodes . . . 17

6.2 Evidence and grading model . . . 17

6.3 Fact normalisation . . . 18

6.4 Scheduled refresh and delta alerts . . . 18

6.5 Prompt-injection defence by structure . . . 18

7. Data Architecture . . . 20

7.1 Principles . . . 20

7.2 Vector design . . . 21
7.3 Retention and archival . . . 21
7.4 Migrations . . . 21
8. Private-Context RAG (Phase B) . . . 22
8.1 Ingestion pipeline . . . 22
8.2 Redaction policy tiers . . . 22
8.3 Chunking . . . 23
8.4 Retrieval and use . . . 23
8.5 Deletion and quality . . . 23
9. API, Authentication and Billing . . . 24
9.1 Endpoint catalogue . . . 24
9.2 Access control . . . 24
9.3 Conventions . . . 24
9.4 Billing: reserve, commit, refund . . . 25
10. AI Layer (LLMOps) . . . 26
10.1 Model routing by tier . . . 26
10.2 Prompt-caching playbook . . . 26
10.3 Guardrails . . . 27
10.4 Evaluation . . . 27
11. Infrastructure and Cloud Operations . . . 28
11.1 Kubernetes layout . . . 28
11.2 Network and egress . . . 28
11.3 Multi-region disaster recovery . . . 28
11.4 Accounts, environments and Terraform . . . 29
11.5 Secrets and identity . . . 29
12. CI/CD and Release Engineering . . . 30
12.1 Pipeline stages . . . 30
12.2 Safe release of agent logic . . . 30
12.3 Supply chain . . . 30
13. Observability and Reliability . . . 31
13.1 Service level objectives . . . 31
13.2 Telemetry design . . . 31
13.3 Metrics catalogue . . . 31
13.4 Alerts and operations . . . 31
14. Security, Privacy and Compliance . . . 32
14.1 Threat model (STRIDE) . . . 32
14.2 OWASP Top 10 for LLM applications . . . 32
14.3 Data-flow and sub-processor register . . . 32
14.4 Compliance programme . . . 33
15. Testing Strategy . . . 34

15.1 Chaos experiments with explicit pass criteria . . . 34
16. Cost Model and Unit Economics . . . 35
16.1 Assumptions . . . 35
16.2 Monthly cost by tier . . . 35
16.3 Sensitivity to prompt-cache performance . . . 35
16.4 Optimisation levers . . . 36
17. Roadmap, Milestones and Team . . . 37
17.1 Critical path and sequencing logic . . . 37
17.2 Team (six engineers) . . . 37
18. Production Starter Code . . . 38
18.1 Repository layout . . . 38
18.2 Tenant-scoped database access . . . 38
18.3 Row-level security migration . . . 39
18.4 Research graph and tool ledger . . . 40
18.5 Time-sliced resumable runner . . . 41
18.6 Orchestration loop . . . 42
18.7 Authentication . . . 43
18.8 Run creation: idempotent, reserved, outbox-safe . . . 44
18.9 Isolation test suite (CI hard gate) . . . 45
18.10 Container, local environment and CI . . . 45
19. Risks, Open Questions and Prototype Plan . . . 47
19.1 Risk register . . . 47
19.2 Build this first (Phase 0 spikes with pass criteria) . . . 47
19.3 Open questions for the product owner . . . 47
Appendix A. Glossary . . . 48
Appendix B. Verification Register . . . 49

> **How to read this document.** This edition keeps the structure of the v1 plan (comparable systems, C4 views, ADRs, data, API, AI, infrastructure, CI/CD, observability, security, testing, cost, roadmap, code) and changes what was wrong, missing, or unsafe. Items that depend on a vendor fact I could not verify are tagged **[VERIFY]** and collected in Appendix B so nobody builds on an unchecked assumption.

1. Executive Summary

Goal. Turn a single-operator AI research skill into a multi-tenant SaaS that takes a target company and returns a **graded, auditable account brief**; keeps it fresh with scheduled refreshes and delta alerts; and, in Phase B, blends the client's private material (CRM notes, call recordings) into an **internal-only** view without ever leaking it into the **client-facing** deliverable.

Core thesis. Three properties decide whether this product can be sold to serious customers: (1) **isolation** that holds even when an LLM agent is manipulated, (2) **durability** so multi-hour runs survive failures without paying twice, and (3) **evidence quality** so every claim is traceable and graded. The architecture is organised around making those three true by construction rather than by convention.

1.1 Ten decisions at a glance

#	Decision	Choice	Why it matters
1	Tenancy model	Pooled Postgres with forced RLS by default; bridge and silo tiers for enterprise	Cheap isolation for most tenants, contractual isolation for the few that demand it
2	Durability owner	LangGraph checkpoint is the single source of truth; Inngest only triggers, schedules and loops slices	One retry owner per failure class; no double-retry storms
3	Vector store	pgvector **halfvec(1536)**, hash-partitioned by tenant, HNSW + iterative scan, hybrid with tsvector, rerank	Keeps RLS isolation; fixes the filtered-ANN under-recall that plain RLS+HNSW causes
4	LLM usage	Tiered routing (planner / extractor / judge) by config, prompt-cache-aware prompt layout	Survives model deprecations; cost is controlled per tier
5	Untrusted content	Quarantined, tool-less extractor produces schema-validated JSON; the planner never reads raw web text	Structural defence against indirect prompt injection
6	Two-output rule	DB roles + visibility labels + independent leak detector	Private data cannot reach a public deliverable even if a prompt fails
7	Cost control	Tool-call ledger, budget guard in-graph, probe-then-escalate refresh	Exactly-once billing for paid calls; refresh cost drops sharply
8	Billing	Reserve / commit / refund wallet ledger, atomic balance check	No overspend, no double charge, full audit trail
9	Regional DR	Mumbai primary, Hyderabad warm standby, Aurora Global (async) with RPO under 60 s, RTO 30 min	Honest targets that a game-day can prove
10	Release safety	Canary with auto-rollback, graph versions pinned per run, eval gate in CI	Prompt and graph changes cannot corrupt in-flight runs

1.2 What changed from v1 (defects found and resolved)

A line-by-line review of v1 found 19 issues. The first eleven are correctness or consistency defects in v1 itself; the rest are gaps that would have hurt in production.

ID	v1 problem	Severity	v2 resolution (where)
D1	**SET LOCAL** only lasts for one transaction; with pooled connections it silently does nothing outside an explicit transaction	Critical	Transaction-scoped **set_config(..., true)** inside **tenant_txn()**; fail-closed policy; pooled-leak test (Ch. 4, 18)
D2	Checkpointer snippet closed its connection, skipped **setup()**, and ignored the autocommit and dict-row requirements	Critical	Dedicated per-run connection with correct options; **setup()** in migrator job (Ch. 5, 18)
D3	**text-embedding-3-large** is 3072-d, not 1536; HNSW on **vector** caps at 2000 dims	High	Request 1536 dims via the model's **dimensions** option and store as **halfvec** (Ch. 7)
D4	halfvec called "binary quantization"	Medium	halfvec is 16-bit float; binary quantization is a different, lossy option (Ch. 7)
D5	Stale model names (3.5 Sonnet/Opus), prices and cost model	High	Tier-based routing, IDs in config, parametrised cost model (Ch. 10, 16)
D6	Aurora Global Database described as near-synchronous	Medium	Replication is asynchronous; RPO and RTO restated as targets (Ch. 11)

ID	v1 problem	Severity	v2 resolution (where)
D7	Pinecone rejected for data leaving the VPC while other SaaS vendors also receive data	High	Data-flow and sub-processor register; self-hosted tracing; honest residency statement (Ch. 11, 14)
D8	Temporal rejected on a faulty determinism argument; Inngest + LangGraph overlap on retries	Medium	Ownership table: one retry owner per failure class; Temporal kept as documented alternative (Ch. 5)
D9	Starter code: wrong Inngest call shape, no run row, ignored the RLS session, stray citation artefact	Low	Rewritten, compile-checked starter code (Ch. 18)
D10	Blue-green ignores in-flight multi-hour runs; checkpoint schema unversioned	High	Canary + graph version pinning + state schema versions (Ch. 12)
D11	Checkpoint RLS assumed a join to **account_runs**, but library tables have no foreign key	High	**thread_id = tenant:run** with a prefix policy; no join needed (Ch. 4)
D12	RLS + HNSW post-filtering returns too few results for small tenants	High	Hash partitions per tenant group + iterative index scans (Ch. 7)
D13	Prompt cache lifetime is short; multi-hour runs lose it between slices	Medium	Longer TTL tier for hot prefixes, warm-up pings, cache-hit SLO (Ch. 10)
D14	"Aggressive" PII redaction would delete the names and contacts that make CRM notes useful	Medium	Per-tenant redaction policy tiers (Ch. 8)
D15	OWASP LLM identifiers mismatched to any single edition	Low	Mapped to the 2025 numbering, edition recorded (Ch. 14)
D16	"Thousands of concurrent runs" asserted without a capacity model	Medium	Little's-law capacity table (Ch. 2)
D17	Multi-hour work inside a single HTTP-triggered step is fragile	High	Time-sliced execution with resume (Ch. 5)
D18	Per-tenant labels on every metric would explode monitoring cardinality and cost	Medium	Tenant dimension in traces and logs; metrics by plan tier (Ch. 13)
D19	Defence against "malicious SQL from the LLM" implied the agent can write SQL	High	Agents get no SQL tool at all; parameterised tools only (Ch. 4, 10)

2. System Requirements

2.1 Product scope

- **Phase A (public research).** Operator or tenant requests a run for a domain. The platform baselines, fans out to brokers, distils and grades facts, then renders a client-facing brief and keeps it fresh.
- **Phase B (enhanced).** Tenant connects private sources. The platform builds a per-tenant, per-account vault and renders a second, internal-only view that blends public and private context.
- **Out of scope for v2.0.** Outbound email sending, CRM write-back beyond approval-gated drafts, and any mobile client.

2.2 Functional requirements

ID	Requirement	Phase
FR-01	Self-serve tenant sign-up, organisation creation, SSO invite	V1
FR-02	Role-based access: admin, seller, viewer	V1
FR-03	Request a run by company name or domain, with entity resolution to a canonical domain	MVP
FR-04	Multi-pass research: baseline, planned fan-out across brokers, distillation, verification	MVP
FR-05	Graded brief: each fact carries grade A to D, confidence, and citations	MVP
FR-06	Explicit states for Blocked, Unverified and Conflicting facts instead of silent omission	MVP
FR-07	Scheduled refresh using a cheap probe that escalates to a full run only on change	V1
FR-08	Delta detection and alert rules (leadership change, job postings, funding, technology change)	V1
FR-09	Alert delivery by email, Slack and signed webhook, with de-duplication	V1
FR-10	Credit wallet, top-up, usage reporting, invoices	V1
FR-11	Private vault ingestion: CRM exports, Drive, Microsoft 365, call recordings	Phase B
FR-12	Two-output rule: public deliverable and internal blended view, never mixed	MVP (public), B (internal)
FR-13	Human approval gate before any outbound or state-mutating action	V1
FR-14	Data export and deletion on request; tenant offboarding with crypto-shredding	V1
FR-15	Admin audit-log viewer	V1
FR-16	Operator console for concierge runs and support	MVP

2.3 Non-functional requirements (measurable)

Attribute	Target	How it is measured
API availability	99.9% monthly (about 43 minutes of budget)	Synthetic probe + ALB success ratio
API latency	p99 under 500 ms for reads and for run creation (run creation returns 202; research is asynchronous)	OpenTelemetry server spans
Run duration	p50 under 10 min, p95 under 45 min, hard ceiling 4 h	run.created to run.completed
Run success	95% of runs reach COMPLETED or PARTIAL within the p95 target	Run status events
Durability	No run lost on pod, node or AZ failure; zero duplicate paid calls	Chaos tests + ledger uniqueness
Isolation	Zero cross-tenant reads or writes	Isolation suite on every build; quarterly penetration test
Disaster recovery	RPO under 60 s, RTO 30 min	Quarterly game-day
Freshness	Signal to alert under 24 h on daily probes (configurable per rule)	Alert latency histogram
Unit economics	Gross margin of at least 70% at list price	Cost per run vs credits charged (Ch. 16)
Security	Encryption in transit and at rest; SOC 2 readiness by month 6	Control evidence collection

2.4 Capacity model

v1 stated that the platform must process "thousands of runs concurrently" without showing the arithmetic. Little's law gives average concurrency as arrival rate times duration. Using a 10-minute mean run and a 10x burst factor for scheduled refreshes:

Tier	Runs / month	Arrivals / hour	Avg concurrent runs	Peak concurrent (10x)
MVP	1,000	1.4	0.2	2
V1	10,000	13.9	2.3	23
Scale	100,000	138.9	23.1	231
Hyper-scale	1,000,000	1388.9	231.5	2315

Implication. Concurrency in the thousands only appears near a million runs per month. Agent workers are I/O bound (waiting on brokers and LLMs), so one worker process can host many in-flight runs. The real scaling limits are **broker rate limits, LLM rate limits, and Postgres connections**, not CPU. The design therefore applies per-tenant and per-broker concurrency caps rather than adding nodes.

3. Architecture Overview

3.1 Design principles

- **Isolation by construction.** The database enforces tenancy; application code is a second layer, never the only one.
- **Fail closed.** Missing tenant context returns zero rows. Missing policy configuration blocks deployment.
- **One owner per concern.** One system owns durability, one owns retries per failure class, one owns billing truth.
- **Evidence first.** No fact without a source; no source without lineage; no grade without a formula.
- **Cost is a feature.** Every paid call is ledgered, capped and attributable to a run and a tenant.
- **Version everything.** Graphs, prompts, schemas and checkpoint state carry versions so changes are safe to roll out and roll back.

3.2 System context

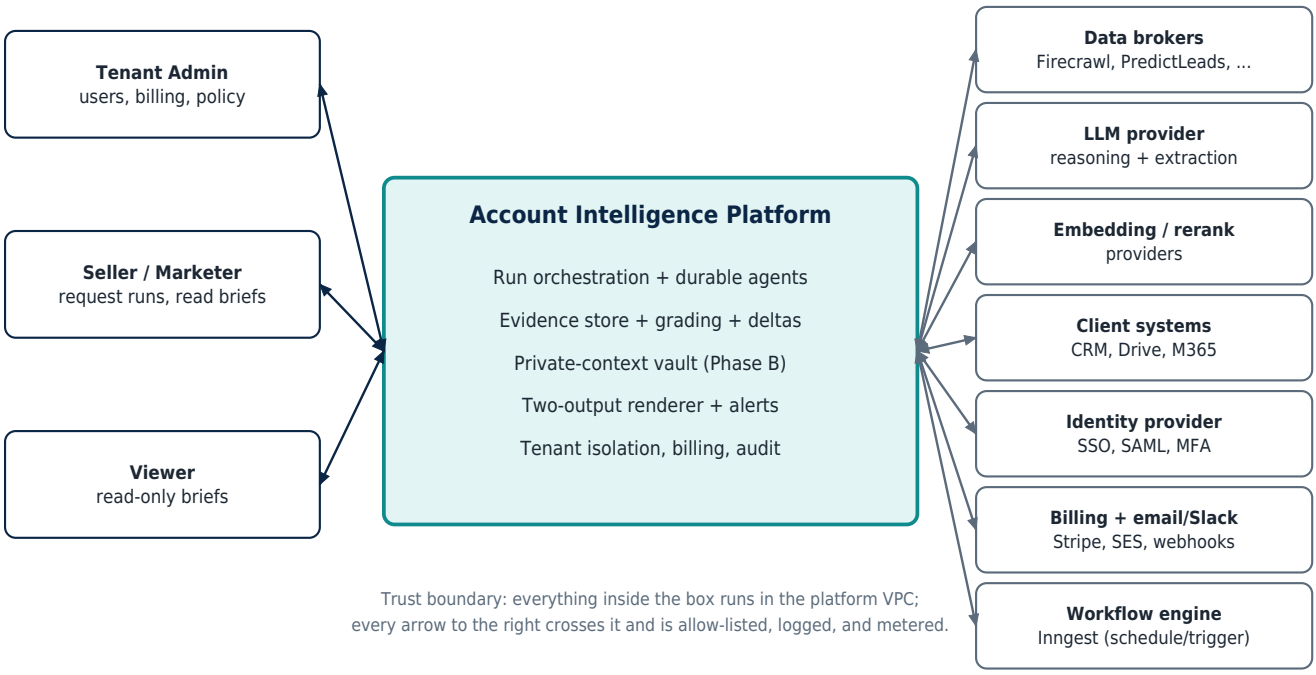


Figure 1. System context. All external dependencies are allow-listed, metered and logged.

3.3 Container view

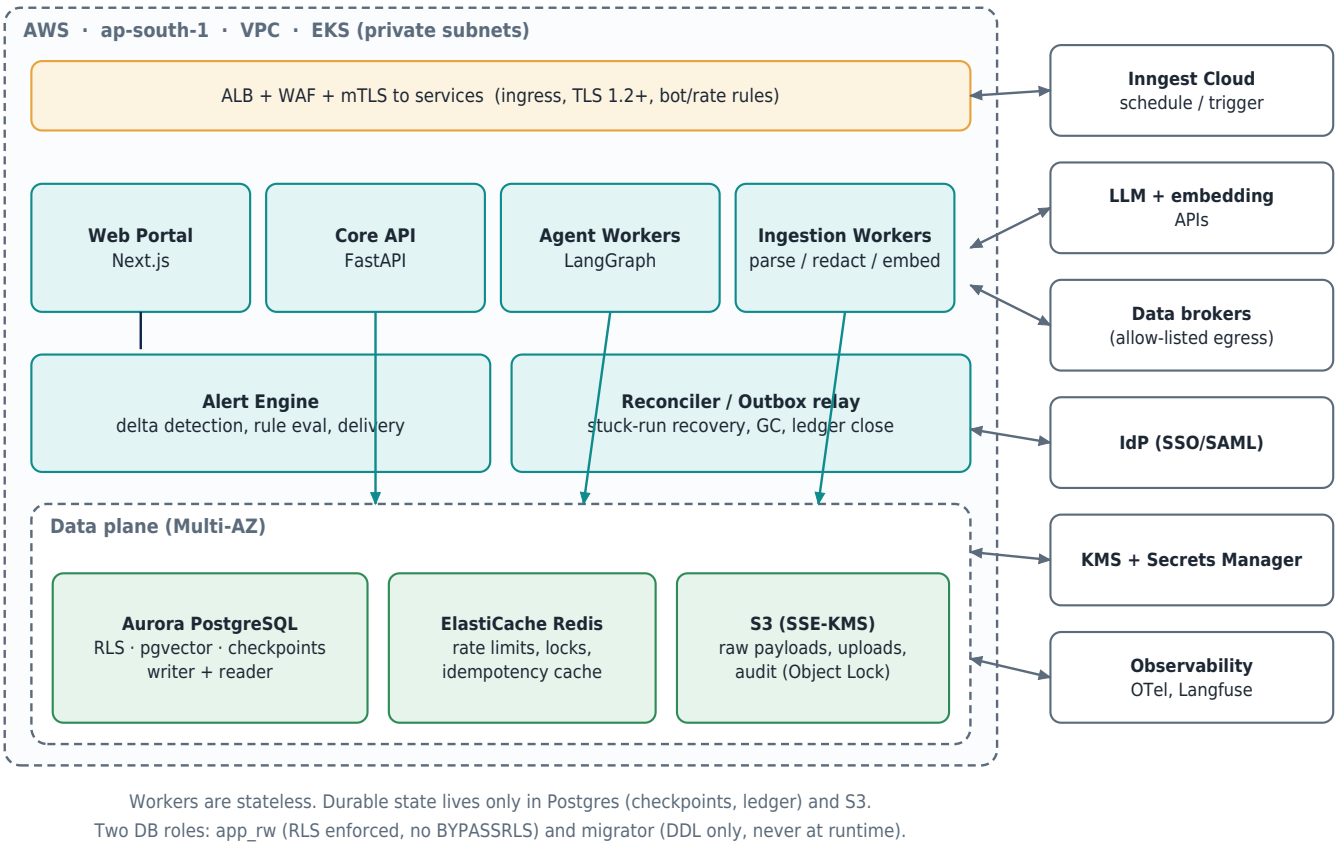


Figure 2. Containers inside the primary region. Workers are stateless; durable state lives only in Postgres and S3.

3.4 Component responsibilities

Component	Responsibility	Scaling signal
Web Portal (Next.js)	Dashboard, brief viewer, alert configuration, admin console	Request rate
Core API (FastAPI)	AuthN/Z, tenant context, run creation, billing reservation, brief retrieval, outbound webhooks	CPU and p99 latency
Agent Workers (LangGraph)	Execute run slices, call brokers and LLMs through the ledger, write facts and checkpoints	In-flight runs and queue age
Ingestion Workers	Virus scan, parse, transcribe, redact, chunk, embed private documents	Documents waiting
Alert Engine	Evaluate fact deltas against rules, de-duplicate, deliver	Deltas per hour
Reconciler	Re-send events for stuck QUEUED runs, settle abandoned reservations, garbage-collect checkpoints	Cron, singleton with lease
Aurora PostgreSQL	System of record: tenants, runs, facts, vectors, checkpoints, ledger, audit	CPU, memory, connections
Redis	Rate limiting, short-lived idempotency cache, distributed locks (never source of truth)	Memory
S3 + KMS	Raw broker payloads, uploads, rendered artefacts, audit archive with Object Lock	Storage

3.5 Architecture decision records (summary)

ADR	Decision	Alternatives considered	Consequence
001	Pooled RLS tenancy with tier upgrade path	Schema per tenant; DB per tenant	Lowest cost; requires disciplined RLS testing; silo tier covers regulated buyers
002	LangGraph checkpoint as durability source of truth	Temporal workflows; Celery	First-class LLM state, time travel, human-in-the-loop; requires our own slice loop
003	Inngest as outer trigger, schedule and loop	Temporal; SQS + cron workers	Fast to build; managed dependency; per-step time limits drive slice design
004	pgvector with halfvec and partitions	Pinecone, Milvus, OpenSearch	One datastore, one isolation model; must manage HNSW memory
005	Quarantine extractor for untrusted text	Single agent with guard prompts	Costs an extra model hop; removes the injection path to tools
006	Visibility labels + DB roles for two-output rule	Prompt-only separation	Structural guarantee; renderers need role switching
007	Reserve, commit, refund wallet ledger	Post-hoc debit from telemetry	No overspend under concurrency; slightly more write traffic
008	Argo Rollouts canary + graph version pinning	Blue-green swap	Safe for long runs; more moving parts in CD
009	Self-hosted Langfuse, OpenTelemetry to Datadog	SaaS LLM tracing	Prompts and outputs stay in our boundary; we operate one more service
010	Tiered model routing by configuration	Hard-coded model IDs	Model upgrades are a config change gated by evals

4. Tenant Isolation Architecture

Isolation is the paramount constraint, so it gets its own chapter. The model is **defence in depth with six layers**; any single layer failing must not produce a cross-tenant disclosure.

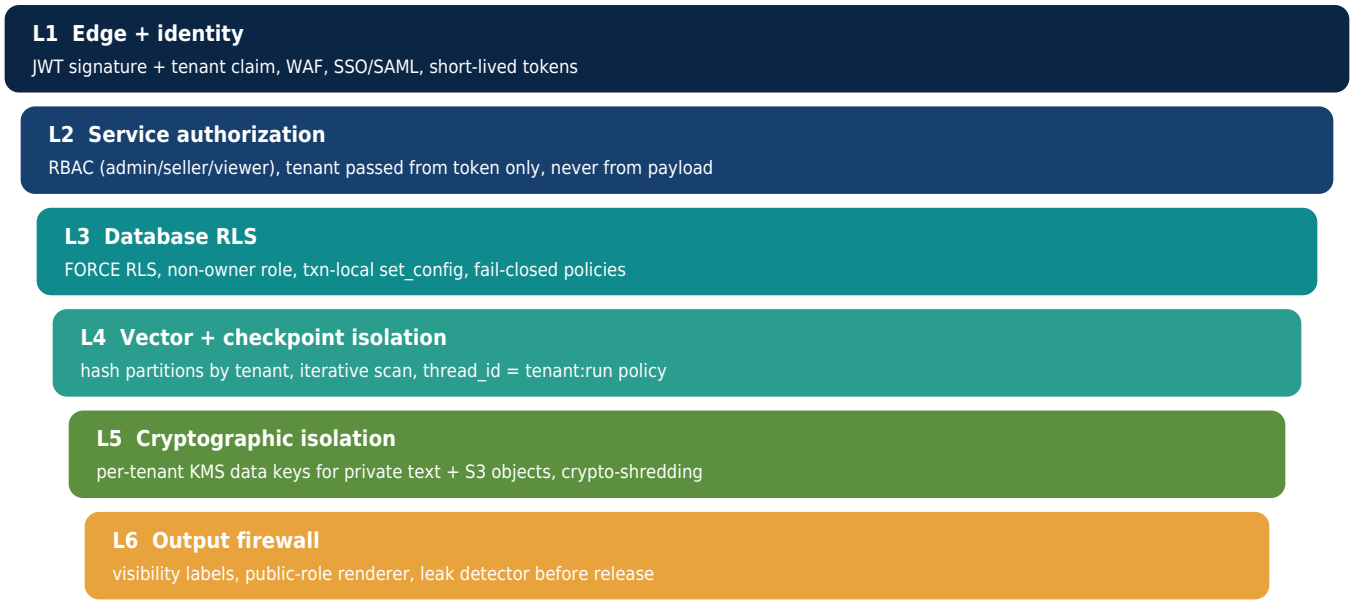


Figure 3. Six isolation layers. L3 to L6 hold even if the application layer (L2) has a bug.

4.1 Isolation tiers

Tier	Mechanism	Who gets it	Extra cost
Pooled	Shared cluster, forced RLS, hash-partitioned vectors, shared KMS key with tenant encryption context	Default for all self-serve tenants	None
Bridge	Dedicated vector partitions and dedicated per-tenant KMS key; same cluster	Mid-market with contractual asks	Low
Silo	Dedicated Aurora cluster (and optionally dedicated node pool and namespace), per-tenant KMS key	Regulated or very large enterprise	High; priced into the plan

4.2 RLS correctness checklist

Row-level security is only as strong as its weakest configuration detail. Every item below is enforced by a test or a migration lint.

Rule	Why	Enforced by
Services connect as a non-owner role without BYPASSRLS	Owners and BYPASSRLS roles skip policies	Migration lint; startup self-check query
FORCE ROW LEVEL SECURITY on every tenant table	Applies policies even to the owner	Migration lint
Tenant set with set_config(..., true) inside an explicit transaction	Transaction-local; cannot leak through the pool	tenant_txn() is the only DB entry point; pooled-leak test
Policy reads a fail-closed accessor (NULLIF to NULL)	Unset or empty value matches no rows rather than erroring or matching all	Unset-context test
Policy uses (SELECT app_tenant()) wrapper	Evaluated once per query, keeps index plans intact	Query-plan regression test
WITH CHECK on every policy	Blocks writing rows into another tenant	Cross-tenant write test
Views are security_invoker; SECURITY DEFINER functions are reviewed and listed	Definer functions run with owner rights	CODEOWNERS on migrations

Rule	Why	Enforced by
Agents have no SQL tool	An LLM must never compose a query, even a filtered one	Tool registry contract test
Cross-tenant jobs (reconciler) use a narrow definer function that returns IDs only	No broad BYPASSRLS service role in the runtime path	Role inventory test

Pooling trap. RDS Proxy and some poolers **pin** a connection when session state is set, which defeats multiplexing. Request-path code uses transaction-local settings, which are compatible with transaction pooling. Workers use a **dedicated** connection per run (Chapter 5) and connect directly to the writer or through a session-mode pooler. Size the writer's max connections as (API pool x pods) + (worker concurrency x pods) + headroom.

4.3 Checkpoint and vector isolation

- **Checkpoints.** The LangGraph library creates its own tables keyed by **thread_id**. v1 assumed a join back to the runs table; there is no foreign key to join on. v2 makes **thread_id = '<tenant_uuid>:<run_uuid>'** and applies a prefix policy to the three checkpoint tables, so the database itself rejects another tenant's thread ids.
- **Vectors.** The **chunks** table is hash-partitioned by tenant. A filtered HNSW scan can return fewer than k rows when the filter removes most candidates; partitions shrink the candidate pool per tenant and iterative index scans keep scanning until enough rows pass the filter. Recall is measured per tenant size band, not globally.
- **Object storage.** Uploaded files live under **tenant_id/account_id/** prefixes with SSE-KMS and an encryption context carrying the tenant id. IAM for workers allows access only through short-lived, prefix-scoped credentials issued per job.

4.4 Cryptographic isolation and offboarding

Private content is encrypted at rest by Aurora and S3 with KMS. Bridge and silo tiers add a per-tenant customer-managed key. Offboarding schedules key deletion after the contractual retention window, which renders any residual backup copies unreadable (**crypto-shredding**). Backups are therefore covered by the deletion promise without rewriting them.

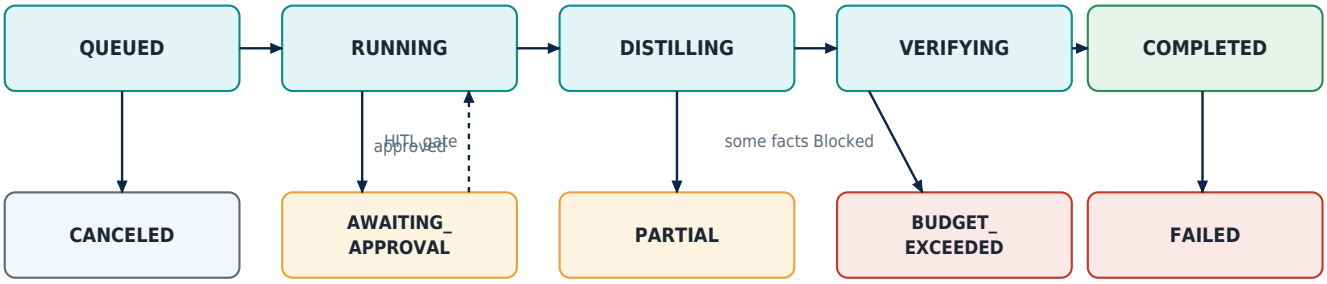
4.5 Attack scenarios the design must survive

#	Attack	First defence	Backstop
A1	Attacker edits the JWT tenant claim	Signature, issuer, audience and expiry verification; membership re-check	RLS binds to the verified tenant only
A2	Injected web text tells the agent to fetch another tenant's data	Quarantine extractor; tools take no tenant parameter at all	RLS returns zero rows; alert on RLS-denied spikes
A3	Agent is coaxed into writing SQL	No SQL tool exists	Non-owner role; statement allow-list at the tool layer
A4	ANN search leaks neighbours across tenants	Partitioned tables plus tenant policy	Per-tenant recall and leakage tests with planted canary vectors
A5	Cache key collision in Redis	Keys always prefixed with tenant id and purpose	Redis holds no authoritative or private data
A6	Private text appears in logs or traces	Structured logging with field allow-list; prompt capture only in self-hosted Langfuse	Log scanner for planted canary strings
A7	Guessed or replayed presigned URL	Short expiry, single object, tenant-prefix scoped, bound to IP where practical	S3 access logs alerting
A8	Support or admin impersonation abuse	Just-in-time access with reason code and customer-visible audit entry	Immutable audit log with Object Lock
A9	Private facts surface in the public deliverable	Public renderer role cannot read private rows	Leak detector blocks release

5. Run Lifecycle and Durability

A run can take minutes to hours, calls paid APIs, and must survive evictions, deployments, and provider outages without paying twice or losing work. This chapter defines the lifecycle, who owns which kind of failure, and how execution is sliced so no single request has to stay alive for hours.

5.1 Run state machine



Terminal states: COMPLETED, PARTIAL, FAILED, CANCELED, BUDGET_EXCEEDED. Each terminal state settles the wallet reservation: commit actual cost, refund the remainder, and release the checkpoint for garbage collection after the retention window.

Transitions are single UPDATE ... WHERE status = expected (optimistic), so duplicate deliveries are harmless.

Figure 4. Run states. Transitions are optimistic single-row updates, so duplicate deliveries are harmless.

5.2 End-to-end sequence

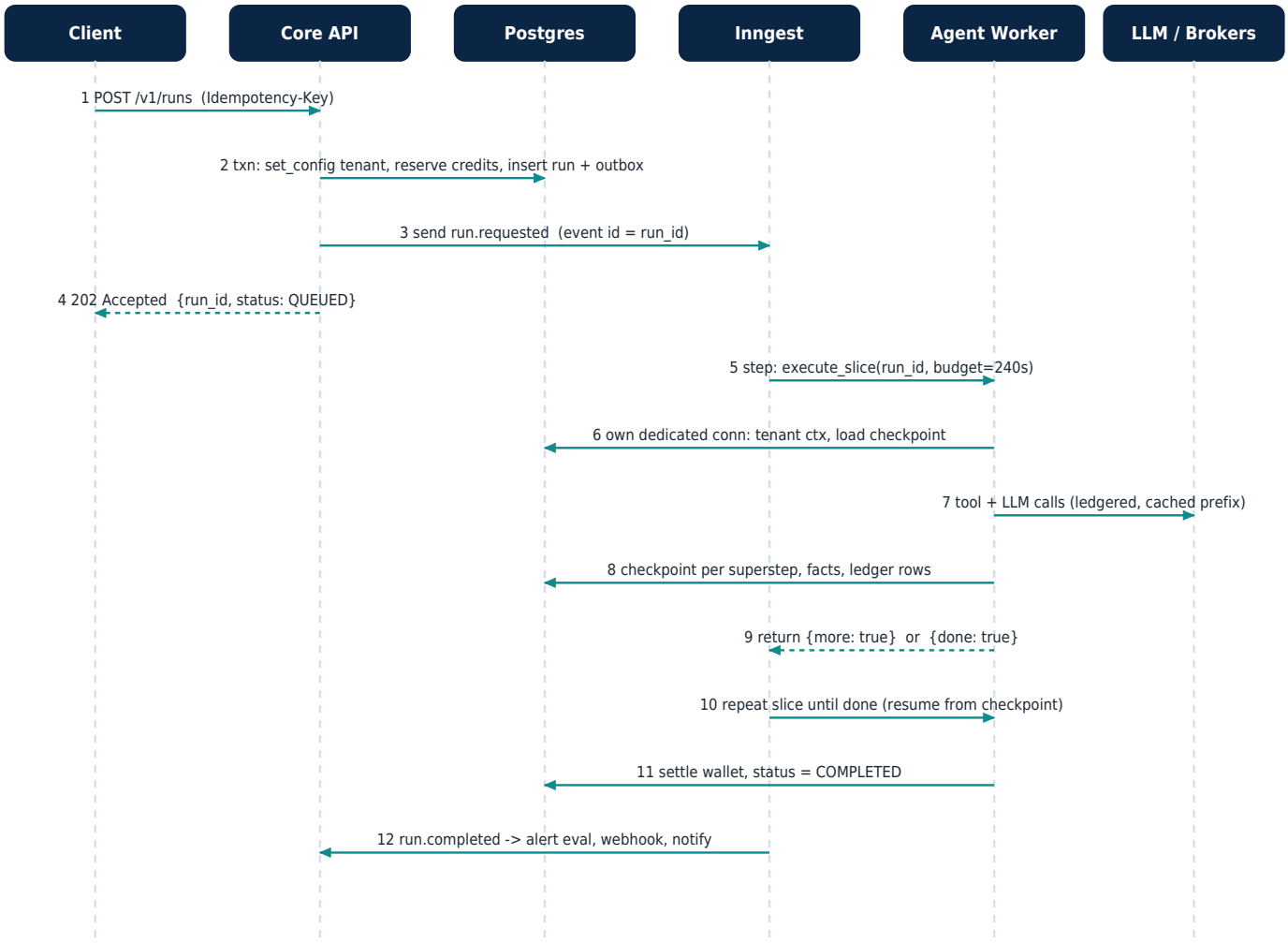


Figure 5. Critical path of a research run, including the slice loop that replaces one long-lived step.

5.3 One owner per failure class

v1 layered Inngest retries on top of LangGraph checkpoints without saying who retries what, which produces retry storms and duplicated spend. v2 assigns every failure class to exactly one owner.

Failure class	Owner	Behaviour
Broker 429 or 5xx, LLM overload	LangGraph node RetryPolicy	3 attempts, exponential backoff with jitter, then mark the fact Blocked (run continues)
Pod eviction, OOM, node loss, deploy rollover	Inngest step retry + checkpoint	Next slice resumes from the last checkpoint; ledger prevents re-billing
Slice time budget reached	Runner	Return more=true; Inngest schedules the next slice
Poison input or schema failure	Node, then run	No retry; fact Blocked with reason; run may finish PARTIAL
Budget exceeded	Budget guard in graph	Stop fan-out, distil what exists, status BUDGET_EXCEEDED
Inngest outage or lost event	Reconciler	Re-sends events for runs stuck in QUEUED beyond 60 s (event id = run id, so delivery is idempotent)
Regional failure	DR runbook	Resume from replicated checkpoints in the standby region

5.4 Time-sliced execution

Inngest functions are driven over HTTP, and each step has platform time and payload limits **[VERIFY current limits]**. Instead of one multi-hour step, the Inngest function loops: each iteration is a short step that calls the worker with a time budget (default 240 seconds). The worker resumes the graph from its checkpoint, runs until the budget expires or the graph ends, and returns. Because LangGraph checkpoints after every superstep, cancelling at the deadline loses at most the node in flight, and the tool ledger makes that node's paid calls free to replay. A hard ceiling of 60 slices gives the 4-hour limit.

- **Human-in-the-loop.** A graph interrupt returns **awaiting=true**; the Inngest function waits for an approval event (24 h timeout) and resumes.
- **Cancellation.** The cancel endpoint sets status CANCELED; the runner checks status between supersteps and exits; the wallet is settled for work already done.
- **Fairness.** Per-tenant concurrency keys prevent one tenant's scheduled refreshes from starving others.

5.5 Tool-call ledger: exactly-once billing

Every paid call (broker or LLM) goes through **ledgered_call()**. The key is a hash of the tool name and canonicalised arguments, unique per run. A replay after a crash finds the stored response and returns it without calling the vendor. Large responses are stored in S3 and referenced by key. The ledger also powers per-run cost attribution, broker spend dashboards, and deterministic re-runs for debugging.

> **Be precise about the guarantee.** The ledger gives exactly-once *billing*, not exactly-once *side effects*. A crash between the vendor call and the ledger insert can still cause one repeat call. Keep paid calls idempotent where the vendor supports idempotency keys, and accept the residual risk for the rest; it is bounded to one call per crash.

5.6 Budgets and circuit breakers

Control	Default	Enforced where
Credits reserved per run	From plan and depth	API, before dispatch
Broker spend cap per run	$5.00 hard	Budget guard before every fan-out
LLM token cap per run	Per depth tier	Wrapper around every model call
Tool calls per run	60	Budget guard
Planner-loop count	2 gap-filling loops	Graph conditional edge
Wall clock	4 hours	Slice counter
Per-tenant daily spend cap	Set by plan	Reservation check
Per-broker concurrency	Vendor-specific	Redis token bucket shared by workers

v1 placed the circuit breaker in the orchestrator, which only observes spend after the fact. v2 enforces caps inside the graph, before the call is made, and uses the orchestrator only as a backstop.

5.7 Version safety

- **Graph version.** Recorded on the run at creation. Workers carry several graph versions at once; a run always resumes on its own version.
- **State schema version.** The state carries **schema_version**. A migration function upgrades old checkpoints on load; removing a state field requires one release of dual support.
- **Prompt versions.** Prompts are packaged and versioned with the graph, never loaded from a mutable store at run time.

6. Research Agent Design

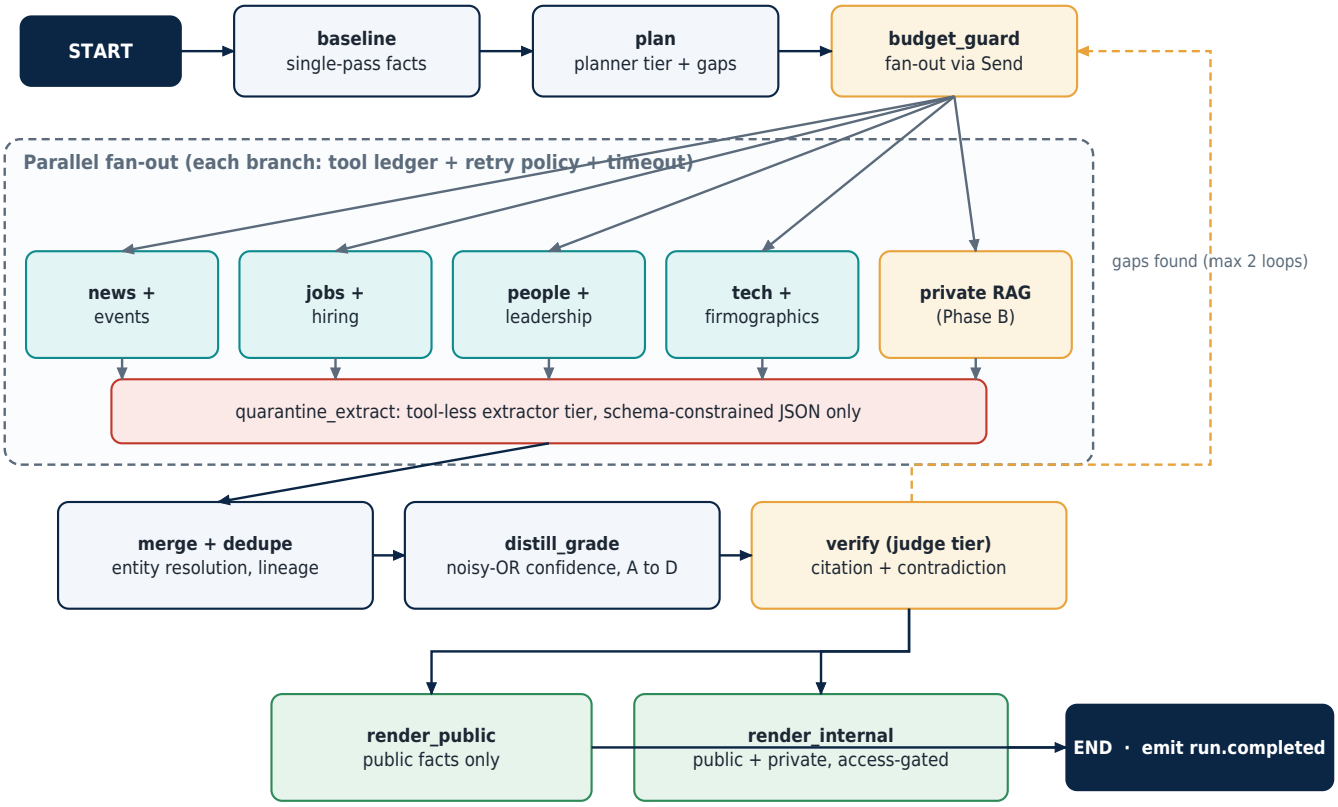


Figure 6. Research graph. Untrusted text is only ever read by the quarantined extractor.

6.1 Nodes

Node	Model tier	Tools	Output	Idempotent
baseline	Extractor	Domain resolver, one cheap overview call	Entity card, initial facts	Yes (ledger)
plan	Planner	None	Typed list of research branches and gaps	Yes
budget_guard	None (code)	None	Allow or stop fan-out	Yes
fetch_branch (x N)	None (code)	One broker adapter each	Raw payload reference	Yes (ledger)
quarantine_extract	Extractor	**None**	Schema-valid JSON facts with quote spans	Yes
merge	None (code)	None	Deduplicated facts with lineage groups	Yes
distill_grade	Planner	None	Graded facts, conflicts flagged	Yes
verify	Judge	None	Citation check, contradiction check, gap list	Yes
render_public / render_internal	Planner	None	Deliverable documents	Yes

6.2 Evidence and grading model

v1 promised "graded facts" without defining the grade. v2 defines it so that grades are reproducible and can be calibrated.

- **Source tiers and weights.** T1 primary (company site, filings, official releases) 0.90; T2 curated structured brokers 0.75; T3 reputable news 0.70; T4 scraped or aggregated 0.40. Weights are starting values, tuned against the golden set.
- **Recency decay.** r = 0.5 ^ (age / half-life), with half-lives by attribute class: hiring signals 21 days, technology 90 days, leadership 180 days, funding 365 days.

- **Lineage groups.** Two brokers that both derive from the same upstream source are one independent source. Within a lineage group take the maximum, not the sum.
- **Confidence (noisy-OR).** c = 1 - product over lineage groups of (1 - w_i x r_i).

Grade	Rule	Display
A	c at least 0.85 and at least two independent lineage groups, one of them T1 or T2	Verified
B	c at least 0.70	Likely
C	c at least 0.50	Indicative
D	c below 0.50	Unverified (shown, clearly labelled)
Conflicting	Two independent groups assert different normalised values, both with c above 0.50	Both values shown with evidence
Blocked	Required source failed or was rate limited after retries	Reason shown; never silently omitted

Worked example. A CTO name seen fresh on the company site (T1, 0.90) and in a curated broker (T2, 0.75), independent lineage: c = 1 - (0.10 x 0.25) = 0.975, grade A. The same name from two aggregators that copy one upstream: one lineage group, c = 0.40 at best, grade D.

Calibration. On the golden set, plot predicted confidence against observed correctness and track expected calibration error. A grade that means "A is right 95% of the time" is a product feature; one that merely sounds confident is a liability.

6.3 Fact normalisation

Facts are stored as entity, attribute key and normalised value (for example **exec.cto.name**), with a **fingerprint** = hash(entity, attribute, normalised value). Normalisation (case, titles, currency, dates, domain forms) happens in code, not in the model, so equal facts always collide and deltas are exact.

6.4 Scheduled refresh and delta alerts

Full runs are expensive, so refreshes use **probe then escalate**. A probe calls only the cheapest high-signal sources (job postings, news and event feeds). It escalates to a full run when the delta score crosses a threshold or when the account's brief is older than its maximum age.

Rule type	Trigger	Noise control
Leadership change	Fingerprint change on a watched role (C-suite, VP of watched functions)	Require two sources or one T1 source
Hiring signal	New posting matching tenant keywords or seniority	Debounce: a vanished posting must be absent on two probes before REMOVED
Funding or M&A;	New funding or acquisition fact at grade B or better	Dedupe on event fingerprint
Technology change	Added or removed technology in the tracked stack	Ignore changes below grade C
News topic	Article matches tenant topics and passes relevance threshold	Per-account daily cap
Confidence shift	Material fact drops a grade	Digest only, never real-time

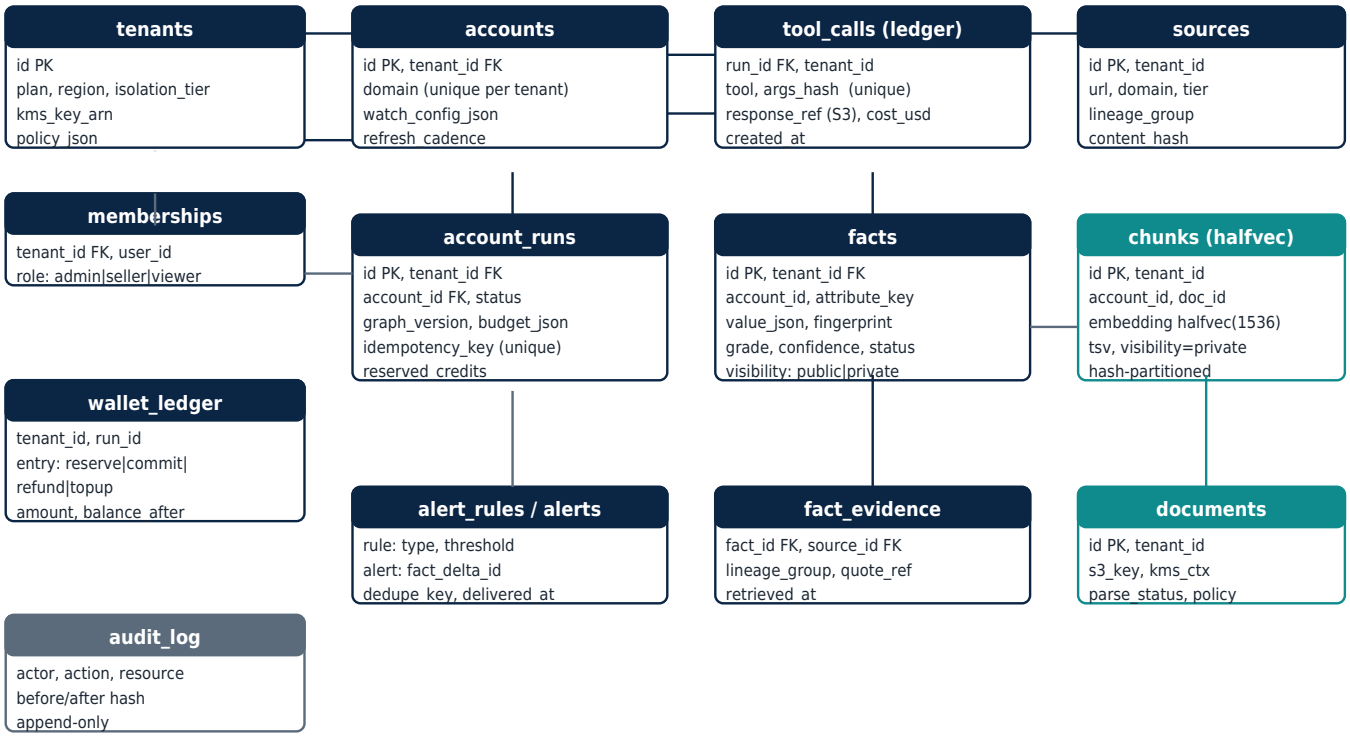
Alerts carry a **dedupe key** (rule, account, fingerprint), are delivered by email, Slack or signed webhook, and are batched into a digest when volume exceeds the tenant's threshold, to prevent alert fatigue. Illustrative economics: if a full run costs about $5.60 and a probe about $0.30, and 15% of probes escalate, the expected refresh cost is about $1.14, roughly 80% below refreshing everything.

6.5 Prompt-injection defence by structure

- **Quarantine.** Raw web pages, job posts and private documents are only ever read by the extractor, which has no tools and must emit JSON matching a schema. The planner and tool-calling nodes see only validated structured facts.
- **Typed tool arguments.** Tools accept typed, validated parameters (a domain matching a pattern, an enum branch), never free text built from untrusted content. No tool accepts a tenant id.
- **Output sanitising.** Extracted strings are length-capped, stripped of control characters, and cited URLs must resolve to the domain the source claimed.

- **Canary and anomaly checks.** The extractor carries a canary instruction it must never follow; any output echoing it flags the source as hostile and drops it.
- **No dangerous capability in the same context as untrusted text.** Nodes that read untrusted content cannot send, write or fetch.

7. Data Architecture



Every tenant-owned table carries tenant_id NOT NULL and an RLS policy on it. Checkpoint tables (library-owned) are keyed by thread_id = '<tenant_id>:<run_id>' and protected by a prefix policy; see Chapter 4.

Figure 7. Logical data model. Teal tables belong to the private-context vault (Phase B).

7.1 Principles

- Every tenant-owned table has **tenant_id NOT NULL**, a foreign key to tenants, and forced RLS with a WITH CHECK clause.
- Facts are append-only with a **superseded_at** marker so deltas and history are reconstructible.
- A wallet table holds the current balance; **wallet_ledger** is the audit trail and a nightly job verifies balance equals the ledger sum.
- Ledger and audit tables are insert-only for the application role (no UPDATE or DELETE grants).

7.2 Vector design

Decision	Choice	Reason
Embedding size	1536 dimensions via the embedding model's dimension option	A 3072-d vector exceeds the 2000-d HNSW limit for **vector**; 1536 keeps most of the quality
Column type	**halfvec(1536)** (16-bit float)	Half the memory; supports HNSW up to 4000 dims. Not binary quantization
Index	HNSW, cosine, m=16, ef_construction=64, ef_search tuned per query class	Handles streaming inserts; tune against recall targets
Filtered search	Iterative index scan enabled (**hnsw.iterative_scan**) **[VERIFY version]**	Keeps scanning until enough rows pass the tenant and account filter
Partitioning	16 hash partitions by tenant_id; large tenants get dedicated partitions	Smaller graphs per tenant, faster builds, easier tenant deletion
Hybrid retrieval	Vector top-50 + full-text top-50, fused with reciprocal rank fusion, then rerank	Names and acronyms need lexical match; embeddings alone miss them
Optional shortcut	Binary-quantised first stage with full-precision re-rank	Only if memory becomes binding; measure recall loss first
Model change	**embedding_model** and **version** columns; dual-write and swap	Re-embedding is a migration, not an incident

Sizing rule of thumb. A halfvec(1536) row is about 3 KB for the vector, plus a similar amount for the HNSW graph and text. Plan 6 to 10 KB per chunk and benchmark. One million chunks is roughly 6 to 10 GB; keep hot partitions inside the buffer cache by choosing the instance class accordingly.

7.3 Retention and archival

v1 pruned checkpoints after 30 days, which keeps far more state than needed. A checkpoint exists to resume a run; once the run is terminal its value is debugging only.

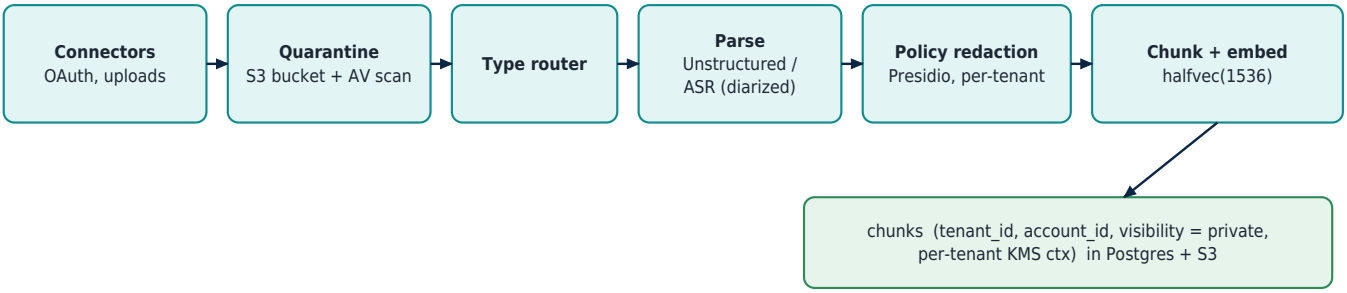
Data class	Hot	Archive	Delete
Checkpoints	Until run terminal + 7 days	None	Then delete the thread
Raw broker payloads	30 days (S3 Standard)	12 months (Glacier tier)	After archive window or on tenant request
Facts and evidence	Current + history for deltas	Superseded versions to Parquet after 24 months	On tenant deletion
Chunks and documents	Until the tenant deletes	None	Cascade delete; propagate to S3 within 24 h
Run metadata	13 months	Parquet in S3	Per policy
Wallet ledger	Permanent	n/a	Never (financial record)
Audit log	90 days queryable	7 years in S3 with Object Lock	Per compliance policy

7.4 Migrations

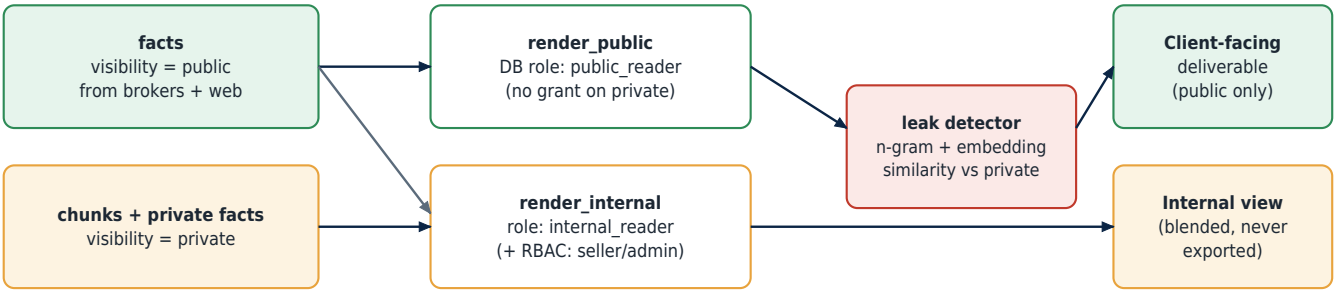
- Alembic, with **expand then contract**: add nullable columns and new tables first, deploy, backfill, then drop. Every migration is backwards compatible with the previous app release.
- The migration job runs as the **migrator** role and also executes the LangGraph checkpointer **setup()**.
- A CI step applies migrations to a scratch database and runs the isolation suite against the result, so a policy regression fails the build.

8. Private-Context RAG (Phase B)

A. Ingestion (per tenant, per account vault)



B. Two-output rule (structural, not prompt-based)



A public deliverable can only be produced by a role that cannot read private rows; the leak detector is a second, independent check.

Figure 8. Private ingestion and the two-output rule.

8.1 Ingestion pipeline

Stage	Design	Notes
Connect	OAuth connectors (CRM, Drive, M365) and presigned multipart uploads direct to S3	Uploads bypass the API so large files cannot hurt API p99
Quarantine	Files land in a quarantine bucket, are virus scanned, size and type checked, then promoted	S3 event triggers ingestion; nothing is parsed before the scan passes
Parse	Document parser for PDFs and office files; speech recognition with speaker diarization for audio **[VERIFY vendor pricing and accuracy]**	Parsers run in a sandboxed pod with no network except the vendor
Redact	Per-tenant policy via Presidio plus custom recognisers	See 8.2
Chunk	By content type (8.3)	Chunk text is prefixed with a short context header
Embed	1536-d embeddings, batched, retried, rate limited per provider	Stored with model and version
Index	Insert into the tenant's partition under tenant context	Visibility is fixed to private by a CHECK constraint

8.2 Redaction policy tiers

v1 redacted PII "aggressively". For sales material that destroys value: contact names, titles and emails are the point of a CRM note. Redaction therefore becomes a tenant-level policy rather than a blanket rule.

Tier	Redacted	Kept	Default for
Strict	All personal identifiers including names and emails	Roles and organisation names	Regulated tenants
Standard	Government IDs, payment cards, health data, credentials, bank details	Business contact names, titles, work emails	Everyone (default)

Tier	Redacted	Kept	Default for
Minimal	Credentials and payment cards only	Everything else	Opt-in with data-processing addendum

8.3 Chunking

- **Documents:** structure-aware (headings, sections), 400 to 800 tokens, 10 to 15% overlap, heading path prepended.
- **Call transcripts:** windows of speaker turns of about 60 to 90 seconds, with speaker labels, call date and opportunity stage as metadata.
- **CRM notes:** one note per chunk, with account, stage, owner and date metadata.
- **Fallback:** v1's fixed 512-token window with 50-token overlap for unstructured text.

8.4 Retrieval and use

- Scope: tenant (RLS) and account (filter) on every query.
- Hybrid fusion then rerank; return the top 8 with document id and character span.
- Private snippets enter prompts only inside the quarantine boundary: documents can contain instructions too, so they are treated as untrusted.
- Every private statement in the internal view carries a citation to its document and span.

8.5 Deletion and quality

- Deleting a document removes its chunks and S3 object, and marks any derived facts for re-evaluation; propagation target 24 hours.
- Retrieval quality is tracked with recall@8, nDCG@10 and citation precision on a per-tenant sample, and with planted canary documents for leakage tests.

9. API, Authentication and Billing

9.1 Endpoint catalogue

Method and path	Purpose	Roles	Notes
POST /v1/runs	Create a run	admin, seller	Requires Idempotency-Key; returns 202
GET /v1/runs/{id}	Status and progress	all	ETag; polling or webhook
POST /v1/runs/{id}/cancel	Cancel a run	admin, seller	Idempotent
GET /v1/accounts	List accounts	all	Cursor pagination
GET /v1/accounts/{domain}/brief	Latest graded public brief	all	Public facts only
GET /v1/accounts/{domain}/brief/internal	Blended internal brief	admin, seller	Phase B; never exported
GET /v1/accounts/{domain}/facts	Facts with evidence	all	Filter by grade, attribute
GET /v1/accounts/{domain}/changes	Deltas since a time	all	Cursor pagination
PUT /v1/accounts/{domain}/watch	Alert rules and cadence	admin, seller	Validated against plan limits
POST /v1/knowledge/uploads	Create presigned upload	admin, seller	Replaces v1's streamed upload
DELETE /v1/knowledge/documents/{id}	Delete a document	admin	Starts cascade
GET /v1/wallet	Balance and ledger	admin	Ledger paginated
POST /v1/wallet/topups	Start checkout	admin	Payment webhook is idempotent on event id
GET /v1/audit	Audit entries	admin	Filter by actor, resource
POST /api/inngest	Orchestrator callback	machine	Signature verified; not part of the public API

9.2 Access control

Capability	Admin	Seller	Viewer
View public brief and facts	Yes	Yes	Yes
View internal blended brief	Yes	Yes	No
Create or cancel runs	Yes	Yes	No
Configure alerts	Yes	Yes	No
Upload or delete private documents	Yes	Upload only	No
Manage users, SSO, billing, redaction policy	Yes	No	No
Read audit log	Yes	No	No

Tokens come from an external identity provider and carry subject, tenant and role claims. The API verifies signature, issuer, audience and expiry, and re-checks membership against the database with a short cache so that removed users lose access within about a minute.

9.3 Conventions

- **Versioning.** Path versioning; additive changes only inside v1; deprecations announced with Sunset headers.
- **Errors.** Problem-details JSON with type, title, status, detail and request id.
- **Idempotency.** Keys scoped per tenant and enforced by a unique constraint, not only by a cache; replays return the original result.
- **Rate limiting.** Token bucket in Redis per tenant and per user, with endpoint cost weights; 429 with Retry-After. Limits scale with plan.
- **Outbound webhooks.** HMAC-SHA256 over timestamp and body, five-minute tolerance, exponential retries for 24 hours, event ids for de-duplication.

OpenAPI excerpt: POST /v1/runs

```
paths:
  /v1/runs:
    post:
      operationId: createRun
      parameters:
        - { name: Idempotency-Key, in: header, required: true,
            schema: { type: string, minLength: 8, maxLength: 64 } }
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [domain]
              properties:
                domain: { type: string, pattern: '^[a-z0-9]([a-z0-9-]*[a-z0-9])?(\.[a-z0-9-]+)+$' }
                depth:  { type: string, enum: [standard, deep], default: standard }
      responses:
        '202': { description: Accepted, content: { application/json: { schema: { $ref: '#/components/schemas/RunRef' } } } }
        '402': { description: Insufficient credits }
        '429': { description: Rate limited }
```

9.4 Billing: reserve, commit, refund

v1 debited the wallet asynchronously from telemetry. Under concurrency that allows overspend: ten parallel requests can each see a sufficient balance. v2 reserves credits atomically at run creation and settles at the terminal state.

Ledger entry	When	Effect
reserve	Run creation, same transaction as the run row	Balance reduced only if sufficient (single conditional UPDATE)
commit	Terminal state	Records actual cost; may be less than the reservation
refund	Terminal state	Returns reservation minus actual; full refund when the platform was at fault
topup	Payment provider webhook	Idempotent on provider event id
adjustment	Support action	Requires reason code and audit entry

- Invariants checked nightly: balance equals the ledger sum; no negative balance (CHECK constraint); every terminal run has exactly one settlement.
- BUDGET_EXCEEDED commits up to the cap and refunds the rest; the customer sees partial results rather than an empty failure.
- Credits are priced by depth tier, not raw tokens, so customers see predictable cost.

10. AI Layer (LLMOps)

10.1 Model routing by tier

v1 hard-coded model names and prices that are already out of date, and its cost model inherited that error. v2 routes by **tier**; concrete model IDs live in configuration, change only through the evaluation gate, and use dated identifiers where the provider offers them.

Tier	Used for	Selection criteria	Illustrative default (config) [VERIFY]
Planner	plan, distill_grade, render	Strong reasoning and tool-planning at moderate cost	Sonnet-class: claude-sonnet-5-5
Extractor	baseline, quarantine_extract, normalisation helpers	Fast, cheap, reliable structured output	Haiku-class: claude-haiku-4-5-20251001
Judge	verify node, offline evaluation	Highest accuracy; must differ from the generator	Opus-class: claude-opus-5-5, used sparingly
Embedding	Chunk and query embeddings	Quality at 1536 dims, residency options	text-embedding-3-large with dimensions=1536
Rerank	Retrieval re-ordering	Latency under 300 ms for 50 candidates	Managed cross-encoder or self-hosted

- **Fallback chain.** On sustained provider errors the router falls back within tier (alternate model or provider) before degrading quality; a degraded run is marked so graders and customers can see it.
- **Rate limits.** A shared token-bucket per model and provider account smooths bursts across workers; scheduled refreshes are jittered across the hour.
- **Provider terms.** Confirm zero-retention and no-training terms for private content before Phase B **[VERIFY]**.

10.2 Prompt-caching playbook

Prompt caching lets the provider reuse a stable request prefix at a steep discount. It is the largest lever on LLM cost, but it fails silently: one changed byte in the prefix and the discount disappears. These are the rules.

Rule	Detail
Prefix order	Tool definitions, then system prompt and standard operating procedures, then stable reference text, then run-specific data, then conversation turns
Tenant-free prefix	The cached prefix contains no tenant data, so it is reusable across every tenant's runs. Tenant and account data sit after the last breakpoint
Deterministic bytes	Tools sorted by name; JSON serialised canonically (sorted keys, fixed separators); no timestamps, request ids, or random ordering in the prefix
Explicit breakpoints	Place cache_control at the end of the static prefix and, in long loops, at the end of the accumulated history; respect the per-request breakpoint limit **[VERIFY]**
Minimum length	Prefixes below the model's minimum cacheable length are not cached; pad the stable SOP block rather than splitting it **[VERIFY per model]**
Lifetime	The default cache lifetime is minutes, not hours. Slices that pause for longer than the lifetime lose the cache. Use the longer-lifetime option for the static prefix and warm it at slice start **[VERIFY TTL options and price multipliers]**
Telemetry	Record cache-read, cache-write and uncached input tokens on every call; alert when the cache-read ratio falls below 70% for cacheable calls

Economics to model with current prices: cache reads are priced at a fraction of base input; cache writes cost a premium over base input. A prefix reused at least twice inside its lifetime is already cheaper than not caching. Chapter 16 parametrises these multipliers.

10.3 Guardrails

Layer	Control
Input	Domain validated against a pattern and resolved before spending; blocked-entity list; private content scanned for credentials before indexing
Model	System prompts static; extractor has no tools; JSON schema enforced; max tokens and temperature fixed per node
Tools	Typed arguments; allow-listed brokers; no SQL, shell or arbitrary URL fetch tools; egress via allow-listed proxy; per-run call caps
Output	Every fact must carry at least one evidence reference; claims without evidence are dropped; URLs must match cited domains
Release	Public renderer role cannot read private rows; leak detector runs before any public artefact is released
Human	Approval gate before any outbound or state-mutating action; approvals are audited

10.4 Evaluation

v1's nightly judge over 50 companies is a good start but under-specified. The golden set **decays**, because real-world facts change; a stale label punishes a correct agent.

- **Golden set.** 50 accounts at MVP growing to 200, stratified by size, industry and region; every label has a capture date and is re-verified quarterly.
- **Judge hygiene.** The judge model differs from the generator, is calibrated against human labels on a sample (track agreement), and uses rubric scoring with cited evidence.
- **Paired comparison.** Compare candidate and baseline on the same accounts and report confidence intervals; small sets make raw averages misleading.
- **Adversarial corpus.** A maintained set of prompt-injection pages and poisoned documents; the pass rate must be 100% for tool misuse and for leakage canaries.

Metric	Gate (initial)	Notes
Precision of grade A and B facts	A at least 0.95; B at least 0.85	Against verified labels
Coverage of expected attributes	At least 0.80	Missing is better than wrong
Citation validity	At least 0.97	Cited span must support the claim
Grade calibration error	At most 0.05	Expected calibration error
Injection and leakage suite	100% pass	Any failure blocks release
Cost per run	Within 10% of baseline	From the ledger
p95 run duration	Within 15% of baseline	On replayed fixtures

A candidate is blocked if any gate fails or if a paired test shows a statistically significant drop on a primary metric. Online, a small share of live runs is shadow-graded and drift is tracked weekly.

11. Infrastructure and Cloud Operations

11.1 Kubernetes layout

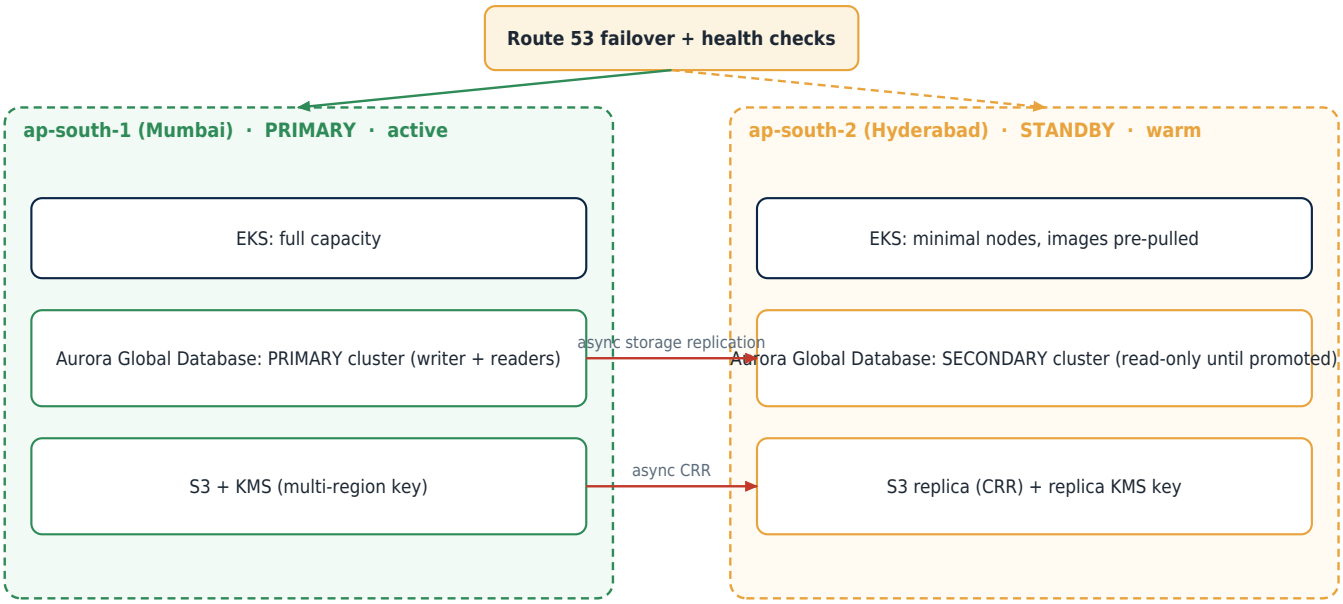
Node pool	Capacity	Workloads	Disruption policy
system	On-demand, 2 small nodes across AZs	Karpenter, ArgoCD, ingress, DNS, observability agents	Never consolidated below 2
api	On-demand, multi-AZ	Core API, web portal, alert engine	PodDisruptionBudget: at most 1 unavailable
agents	Spot first, on-demand fallback	Agent workers	Resumable by design; 120 s grace period; consolidation allowed
ingestion	Spot, memory-optimised if needed	Parsers, embedders	Retry-safe jobs; consolidation allowed
jobs	On-demand, labelled do-not-disrupt	Migrations, reconciler	Voluntary disruption blocked while running

- **Graceful shutdown.** On SIGTERM a worker stops taking slices and exits within the grace period; the checkpoint plus ledger makes interruption safe. Spot reclamation notices are typically two minutes, which is enough.
- **Karpenter.** Replaces the static node-group autoscaler, provisions right-sized capacity from pending pod requests, and consolidates under-used nodes. v1's 30 to 50% saving is plausible but must be measured on our workload, not promised **[VERIFY]**.
- **Right-sizing.** Workers are I/O bound: request small CPU, moderate memory, high concurrency per pod; scale on queue age and in-flight runs rather than CPU.

11.2 Network and egress

- Private subnets for workloads; public subnets only for the load balancer and NAT. VPC endpoints for S3, ECR, STS, KMS and Secrets Manager reduce NAT traffic and exposure.
- Default-deny NetworkPolicies per namespace; explicit allow rules per service pair.
- All outbound fetches go through an **egress proxy** with a domain allow-list and SSRF protection: block link-local and private ranges, cap redirects, and refuse the instance metadata address.
- Parsers for private documents run in a separate namespace with no route except to the document vendor.

11.3 Multi-region disaster recovery



RPO target: under 60 s (async). RTO target: 30 min, proven by quarterly game-day. In-flight runs resume from checkpoints.

Figure 9. Active-passive regions. Replication is asynchronous, so targets are stated as RPO and RTO, not as zero loss.

- **Correction to v1.** Aurora Global Database replicates asynchronously at the storage layer with typically low lag. It is not synchronous, so a regional loss can drop the most recent seconds of writes. Confirm that the chosen Aurora version and the secondary region are supported **[VERIFY]**. The inter-region latency quoted in v1 (about 14 ms) is unverified; measure it.
- **Failover runbook.** (1) Declare regional incident. (2) Promote the secondary Aurora cluster. (3) Scale the standby node pools. (4) Switch DNS. (5) Run the reconciler to re-send events for runs that were queued. (6) Verify the isolation suite against the new writer. (7) Re-point observability. Each step is scripted and rehearsed.
- **What can be lost.** Ledger and checkpoint rows from the last replication-lag window. Consequence: at most a few paid calls may be repeated. The wallet is reconciled against provider invoices after failover.
- **Dependencies.** Confirm the workflow engine's failover story and that event ids de-duplicate across regions **[VERIFY]**. Replicate container images, secrets and KMS keys to the standby region ahead of time.
- **Cost note.** A warm standby roughly doubles database cost; DR starts at V1, not MVP. MVP relies on multi-AZ and tested backups.

11.4 Accounts, environments and Terraform

- AWS Organization with separate accounts: management, security and log archive, shared services (registry, DNS), development, staging, production. Service control policies deny disabling logging and encryption.
- Terraform layout: modules for network, EKS, Aurora, S3 with KMS, IAM roles, observability; live configurations per environment and region; remote state with locking; plans reviewed in pull requests; drift detection nightly.
- Same container digest is promoted from staging to production; only configuration differs.

11.5 Secrets and identity

- Secrets live in Secrets Manager (replicated to the standby region) and reach pods through the External Secrets operator or the CSI driver, preferably **mounted as files** rather than environment variables, which leak into crash dumps and process listings.
- Workloads use EKS Pod Identity or IRSA, with one narrowly scoped role per workload. No long-lived cloud keys.
- Customer-supplied tokens (CRM OAuth) are encrypted with the tenant's KMS context and are never logged.
- Rotation: database credentials automatically; broker keys on a calendar and on personnel change.

12. CI/CD and Release Engineering

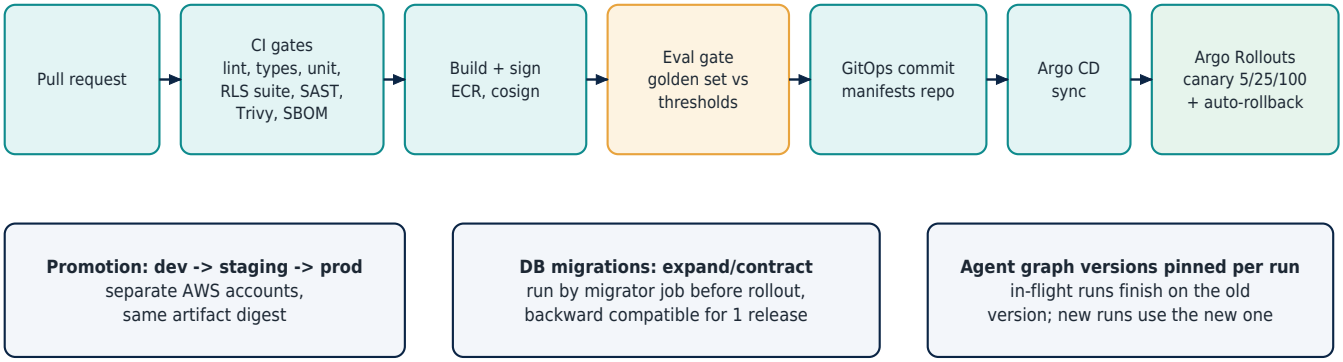


Figure 10. Delivery pipeline. The isolation suite and the eval gate are hard gates.

12.1 Pipeline stages

Stage	Checks	Blocks on
Pull request	Lint, type checks, unit tests, migration lint (FORCE RLS, WITH CHECK, no BYPASSRLS), secret scan	Any failure
Isolation suite	Cross-tenant read, write, unset context, pooled leak, checkpoint prefix, partition leakage with canary vectors	Any failure
Security	SAST, dependency audit, container scan, SBOM generation	High or critical findings
Build	Reproducible image, signed with cosign, pushed to the registry by digest	Signature failure
Eval gate (main)	Golden-set evaluation, injection and leakage corpus, cost and latency on fixtures	Gates in section 10.4
Deploy staging	Smoke tests and synthetic runs using recorded broker fixtures	Smoke failure
Canary prod	5 percent, then 25 percent, then 100 percent with automated analysis	Analysis failure triggers rollback

12.2 Safe release of agent logic

v1 used blue-green with a one-hour rollback window. That does not address runs that last hours and hold state in the old version's format. v2 combines canary rollout with **version pinning**:

- New traffic is routed gradually by Argo Rollouts, with analysis on error rate, p99 latency, run failure ratio, cache-read ratio and cost per run against the stable version.
- Each run is pinned to the graph version it started on; old and new workers coexist until the old version has no active runs, then it is retired.
- Rollback is a routing change plus a version flag, not a data migration. Checkpoint schema changes follow expand and contract.
- Prompts and graph code ship together as one artefact. Feature flags control exposure by tenant, plan or percentage.

12.3 Supply chain

- Third-party actions and base images are pinned by digest; dependency update bots open reviewed pull requests.
- Signatures are verified at admission so unsigned images cannot run in the cluster.
- SBOMs are stored per release and attached to the compliance evidence.

13. Observability and Reliability

13.1 Service level objectives

SLI	SLO	Window	Alerting
API availability (non-5xx / total)	99.9%	30 days	Multi-window burn rate: page at 14.4x over 1 h and 6x over 6 h
API latency p99	Under 500 ms	30 days	Page on sustained breach, ticket on drift
Run success within target	95% reach COMPLETED or PARTIAL within p95 target	30 days	Page on fast burn
Alert freshness	95% delivered within 24 h of the signal	30 days	Ticket
Prompt-cache read ratio	At least 70% on cacheable input	7 days	Ticket; cost regression
Isolation violations	Zero	Always	Page on any RLS-denied spike or canary leak
Checkpoint resume success	99.9% of resumes succeed	30 days	Page on fast burn

13.2 Telemetry design

- **Standards.** OpenTelemetry across API, workers and jobs. The run id is the trace id for the whole run, so the slice loop appears as one trace.
- **Spans.** One span per graph node and per tool or model call, with attributes for tier, model, token counts, cache tokens, cost, and outcome.
- **LLM tracing.** Self-hosted Langfuse (or equivalent) captures prompts and outputs inside our boundary; Datadog receives metrics and scrubbed spans.
- **Logs.** Structured JSON with a field allow-list. Prompts, document text and tokens are never logged. Planted canary strings verify this in tests.
- **Cardinality.** v1 proposed per-tenant dashboards. Putting tenant id on every metric multiplies series count and cost. Tenant and run dimensions live in traces and logs; metrics are aggregated by plan tier, region and tool, and a tenant-level view is built by querying traces on demand.

13.3 Metrics catalogue

Area	Metrics
Runs	Created, completed by status, duration histogram, slices per run, queue age, stuck-run count
Cost	Cost per run by tier, broker spend by vendor, tokens by model, cache read, write and miss, reservation vs actual
Quality	Share of facts by grade, Blocked share, Conflicting share, evidence per fact, judge score on shadow samples
Dependencies	Broker latency and error ratio, LLM latency, time to first token, rate-limit hits
Data	Checkpoint size, table bloat, connection saturation, replication lag, HNSW recall probes
Security	RLS-denied counts, auth failures, injection detections, egress denials

13.4 Alerts and operations

Severity	Examples	Response
Page	Isolation violation, API burn rate, replication lag over 60 s, run success burn, worker crash loop	On-call within 15 minutes
Ticket	Cache ratio decline, broker error rise, checkpoint growth, cost per run drift	Next business day
Digest	Weekly quality and cost review	Engineering review

Every page has a runbook; incidents follow a blameless post-incident review with action items tracked to closure. The DR game-day and a chaos exercise run quarterly.

14. Security, Privacy and Compliance

14.1 Threat model (STRIDE)

Threat	Example	Control
Spoofing	Forged token, stolen API key	Signature and audience checks, short-lived tokens, MFA for admins, key hashing and scoping
Tampering	Altered ledger or facts	Insert-only grants on ledger and audit; hash of before and after state; signed webhooks
Repudiation	Admin denies a deletion	Immutable audit with Object Lock and actor identity
Information disclosure	Cross-tenant read, private text in public brief	Six isolation layers (Chapter 4), leak detector, log scrubbing
Denial of service	Run floods, token-burning prompts	Rate limits, budgets, per-tenant concurrency, circuit breakers
Elevation of privilege	Agent obtains a write tool via injection	Quarantine, typed tools, approval gates, least-privilege IAM

14.2 OWASP Top 10 for LLM applications

Mapped to the **2025 edition numbering**; re-verify if a newer edition is adopted **[VERIFY]**. v1's labels did not match any single edition.

ID	Risk	Control in this design
LLM01	Prompt injection	Quarantine extractor, typed tool arguments, canary checks, no dangerous tools beside untrusted text
LLM02	Sensitive information disclosure	Visibility labels, public renderer role, leak detector, tenant redaction policy
LLM03	Supply chain	Pinned and signed images, SBOMs, vendor register, model and prompt versioning
LLM04	Data and model poisoning	Source tiers and lineage, adversarial corpus, quarantine of uploaded documents
LLM05	Improper output handling	Schema validation, sanitised strings, URL checks, no model output executed or interpolated into queries
LLM06	Excessive agency	No SQL or shell tools, approval gates, per-run call caps, scoped IAM
LLM07	System prompt leakage	No secrets in prompts; prompts treated as non-confidential and versioned
LLM08	Vector and embedding weaknesses	Partitioned, RLS-protected vectors, canary vectors, per-tenant recall and leakage tests
LLM09	Misinformation	Grades, citations, conflict display, judge verification
LLM10	Unbounded consumption	Reservation wallet, in-graph budget guard, rate limits, backstop circuit breaker

14.3 Data-flow and sub-processor register

v1 rejected one vendor because data would leave the VPC while sending data to several others. The honest position is to **know and minimise** every outbound flow, not to claim none.

Recipient	Data sent	Minimisation and controls
LLM provider	Prompts built from public facts; private snippets for the internal view	Zero-retention and no-training terms **[VERIFY]**; private content only inside quarantine; region selection where offered
Embedding and rerank providers	Chunk text and queries	Same terms; consider self-hosted rerank; redaction before send
Speech recognition vendor	Call audio	Processed in batch, deleted after transcription per agreement **[VERIFY]**
Data brokers	Company domain and name only	No tenant or private data ever sent
Workflow engine	Event payloads	**Events carry identifiers only, never content**; payload size limits enforced by tests
Observability SaaS	Metrics and scrubbed spans	Field allow-list; no prompts, document text or tokens
LLM tracing	Prompts and outputs	Self-hosted inside the VPC

Recipient	Data sent	Minimisation and controls
Payments and email	Billing identity, notification addresses	Standard processor agreements

14.4 Compliance programme

- **SOC 2.** Start evidence collection at MVP; Type I around month 6, Type II after the observation window.
- **Privacy law.** The product handles business contact data about identifiable people. Review obligations under India's Digital Personal Data Protection Act and, where applicable, GDPR and similar laws; maintain records of processing, a lawful-basis analysis, notice and objection handling, and DSAR workflows. **Obtain legal advice; this document is not legal advice.**
- **Broker and scraping terms.** Confirm that broker licences and website terms permit the intended commercial use and resale of derived data.
- **Audit trail.** Every mutation records actor, action, resource, time and a hash of before and after state, written to an append-only table and archived under Object Lock.
- **Testing.** Annual external penetration test plus a focused RLS and prompt-injection assessment before the first enterprise customer.

15. Testing Strategy

Layer	What	Tools and approach	Gate
Unit	Normalisation, grading maths, budget guard, ledger keys, state migrations	pytest, hypothesis for property tests on grading and fingerprints	80% line coverage, 100% on grading and billing
Integration	API with real Postgres and Redis, RLS policies, ledger idempotency	Testcontainers, applying the real migrations	All pass
Isolation	Cross-tenant read, write, unset context, pooled leak, checkpoint prefix, partition canaries	Dedicated suite, runs against migrated schema	**Zero tolerance**
Contract	Broker and provider adapters	Recorded cassettes plus nightly live smoke with spend caps	No schema drift
Agent evals	Quality, calibration, injection, leakage, cost	Golden set, adversarial corpus, judge	Section 10.4
End to end	Sign-up to brief to alert	Playwright against staging with fixtures	Critical paths green
Load	API p99 and queue behaviour at peak factor	k6 for API; synthetic run injector with stub brokers	Meets SLOs at 3x forecast
Chaos	Pod kill mid-slice, node loss, Redis loss, Inngest outage, DB failover	Scheduled experiments in staging, then production at low blast radius	No lost run, no duplicate paid call
Security	DAST, dependency audit, injection red-team, RLS bypass attempts	Automated plus external test	No high findings open

15.1 Chaos experiments with explicit pass criteria

Experiment	Method	Pass criterion
Kill worker mid-slice	Delete the pod during fan-out	Run completes; ledger shows no duplicate vendor call for ledgered keys
Lose the workflow engine	Block egress to it for 10 minutes	Reconciler re-sends; no run stuck beyond 5 minutes after recovery
Database failover	Force writer failover	In-flight slices retry and resume; no corrupted checkpoint
Spot reclamation burst	Terminate most agent nodes	Runs resume; p95 duration within 2x target
Poisoned page	Serve a page with injected instructions	Fact dropped or sanitised; no tool misuse; alert raised
Cache invalidation	Inject a timestamp into a prefix in a canary build	Cache-ratio alert fires within 1 hour

Test data policy. Tests use synthetic tenants and generated documents; production data never enters lower environments. Planted canary strings and vectors are present in every test tenant.

16. Cost Model and Unit Economics

v1's table mixed stale model prices with a flat broker assumption and no stated volumes per line. v2 is a **parametrised model**: every input is listed, so it can be recomputed the day real prices and telemetry arrive. Numbers below are **illustrative**.

16.1 Assumptions

Parameter	Value used	Status
Planner-tier price	$3 in / $15 out per million tokens	Illustrative [VERIFY]
Extractor-tier price	$1 in / $5 out per million tokens	Illustrative [VERIFY]
Cache read price multiplier	0.10 of base input	Illustrative [VERIFY]
Tokens per standard run	Planner 250k in / 25k out; extractor 300k in / 40k out	Calibrate from MVP telemetry
Cache-read ratio (planner input)	75%	Target; alert below 70%
Broker cost per run	$4.50 MVP, $4.00 V1, $3.25 at scale (volume pricing)	Replace with contract rates
Embedding and rerank	$0.05 per run when Phase B is on	Illustrative
Speech recognition	$0.26 per audio hour; 10% of runs add one hour	Illustrative [VERIFY]
Infrastructure	Estimates from instance sizes and multi-AZ minimums	Replace with a Terraform-based pricing calculation

16.2 Monthly cost by tier

Cost line	MVP (1k runs)	V1 (10k runs)	Scale (100k runs)
LLM	$1,119	$11,188	$111,875
Brokers	$4,500	$40,000	$325,000
Embedding + rerank	$50	$500	$5,000
Speech recognition	$26	$260	$2,600
Compute (EKS)	$450	$1,100	$4,000
Database (Aurora)	$350	$1,800	$6,500
Observability	$300	$1,500	$9,000
Misc (S3, network, KMS, WAF, workflow)	$250	$900	$4,500
Total per month	**$7,045**	**$57,248**	**$468,475**
Cost per run (all-in)	$7.04	$5.72	$4.68
Broker share of total	64%	70%	69%
Price per run for 70% gross margin	$23.48	$19.08	$15.62

Per-run LLM cost under these assumptions is about **$1.12** (planner $0.62, extractor $0.50), far below the broker cost per run. **Brokers dominate**, not tokens, which changes where optimisation effort pays off.

16.3 Sensitivity to prompt-cache performance

Cache-read ratio	LLM cost per run	Change vs 75%
0%	$1.62	+45%
50%	$1.29	+15%
75%	$1.12	+0%
90%	$1.02	-9%

Where to optimise. Because brokers are the largest cost, the highest-value levers are (1) **probe then escalate** refresh (Chapter 6), (2) per-account caching of broker payloads across tenants only where licences allow it, (3) negotiating volume pricing against the ledger's real spend data, and (4) not re-fetching attributes whose half-life has not expired. Caching and model tiering help, but cannot rescue a broker-heavy cost structure.

16.4 Optimisation levers

Lever	Effect	Risk
Probe then escalate refresh	Cuts refresh cost by roughly 80% when most accounts are unchanged	Missed changes if probes are too narrow; tune thresholds
Half-life based freshness	Skip fetching attributes still fresh	Needs accurate half-lives
Cache-aware prompt layout	Large LLM saving on multi-turn loops	Silent invalidation; monitored by ratio SLO
Extractor tier for bulk work	Most tokens at the cheap tier	Quality; guarded by evals
Spot instances via Karpenter	Lower compute cost	Interruptions; designed for resumption
Cross-tenant sharing of public broker payloads	Avoid paying twice for the same public fact	Licence terms; never applies to private data
Reserved capacity for the database	Lower steady-state cost	Commitment risk

17. Roadmap, Milestones and Team

v1 started with infrastructure. v2 starts with a short **Phase 0 that proves the riskiest assumptions** before committing months of build to them.

Phase	Weeks	Scope	Exit criteria
0 Prove	1 to 3	Four spikes: tenant isolation with forced RLS and pooled connections; slice-and-resume with the tool ledger; prompt-cache ratio on real prompts; broker cost per run on 20 real accounts	All four spike pass criteria met (section 19.2) or the design is amended
1 MVP (concierge)	4 to 11	Terraform baseline, EKS, Aurora, Karpenter; durable graph with grading; operator console; wallet ledger; evidence UI; golden set v1; observability baseline	Operators run 100 accounts end to end; grade calibration measured; zero isolation test failures
2 V1 (self-serve)	12 to 19	SSO and tenant onboarding; RBAC; billing and top-ups; scheduled probes and delta alerts; webhooks; warm standby region and first game-day; SOC 2 evidence collection	10 paying tenants; SLOs met for 30 days; DR rehearsal passed
3 Scale (Phase B)	20 to 28	Private vault: connectors, parsing, redaction tiers, hybrid retrieval, rerank, internal view, leak detector; DSAR flows; bridge and silo tiers	Leakage canaries at zero; retrieval recall targets met; SOC 2 Type I issued or scheduled

17.1 Critical path and sequencing logic

- Isolation and durability come first because every later feature inherits them; retrofitting either is far more expensive.
- Billing ledger precedes self-serve, because free runs with real broker costs are the fastest way to lose money.
- The evaluation harness is built alongside the first graph, not after it, so every change has a baseline.
- Phase B is last because it adds the highest-risk data class; it should land on a platform whose isolation has already been exercised in production.

17.2 Team (six engineers)

Role	Count	Owns
Principal architect / tech lead	1	Isolation model, data design, security reviews, cross-team decisions
Backend engineers (Python)	2	API, billing ledger, broker adapters, workflow integration, alert engine
AI / ML engineer	1	Graph design, grading, prompts and caching, evals, retrieval quality
Frontend engineer	1	Portal, evidence and brief viewer, admin console
Platform / DevOps engineer	1	Terraform, EKS, CI/CD, observability, DR drills

Add a part-time security engineer or fractional consultant from Phase 2 for penetration testing and SOC 2 control design.

18. Production Starter Code

The listings below replace v1's snippets. They are the load-bearing pieces only: tenant context, forced-RLS schema, resumable runner, ledgered tool calls, run creation, orchestration loop, isolation tests, container and CI files. The Python files were compile-checked; they have **not** been executed against live services, so treat SDK call shapes as **[VERIFY]** against the versions you pin.

18.1 Repository layout

Monorepo

```
account-intelligence/
|-- .github/workflows/ci.yml          lint, migrate, isolation gate, tests, scan, eval gate
|-- deploy/
|   |-- terraform/                    modules: network, eks, aurora, s3-kms, iam, observability
|   `-- helm/                         charts: api, agent-worker, ingestion, alert-engine, reconciler
|-- migrations/                       alembic (expand/contract); 0002_rls.sql
|-- src/
|   |-- api/        main.py, auth.py, runs.py, accounts.py, wallet.py, knowledge.py
|   |-- core/       config.py, logging.py, errors.py
|   |-- db/         session.py (tenant_txn), models.py
|   |-- orchestrator/  functions.py (Inngest: trigger, slice loop, waits)
|   |-- agents/     graph.py, runner.py, grading.py, normalise.py, prompts/, tools/
|   |-- ingestion/  connectors/, parse.py, redact.py, chunk.py, embed.py
|   `-- render/     public.py (public_reader role), internal.py, leakcheck.py
|-- tests/
|   |-- isolation/  test_rls.py                    <- hard CI gate
|   |-- unit/  integration/  contract/  e2e/  chaos/
|   `-- eval/       golden/, adversarial/, judge.py
|-- Dockerfile  compose.yml  pyproject.toml  uv.lock
```

18.2 Tenant-scoped database access

src/db/session.py

```
"""src/db/session.py - tenant-scoped database access. All tenant queries go through tenant_txn()."""
from __future__ import annotations

import os
from contextlib import asynccontextmanager
from typing import AsyncIterator
from uuid import UUID

from sqlalchemy import text
from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker, create_async_engine

# DATABASE_URL = postgresql+asyncpg://app_rw:***@aurora-writer/ai  (role is NOBYPASSRLS, not owner)
engine = create_async_engine(
    os.environ["DATABASE_URL"],
    pool_size=int(os.getenv("DB_POOL_SIZE", "10")),
    max_overflow=int(os.getenv("DB_MAX_OVERFLOW", "5")),
    pool_pre_ping=True,
    connect_args={"server_settings": {"application_name": "ai-api"}},
)
SessionLocal = async_sessionmaker(engine, expire_on_commit=False)


@asynccontextmanager
async def tenant_txn(tenant_id: UUID) -> AsyncIterator[AsyncSession]:
    """One transaction == one tenant context.

    set_config(..., true) is transaction-local: when the transaction ends, the setting is reset,
    so a pooled connection can never carry tenant A into tenant B's request. (A bare SET LOCAL
    outside an explicit transaction is a no-op, which is why v1's middleware was unsafe.)
    """
    async with SessionLocal() as session:
        async with session.begin():
            await session.execute(
                text("SELECT set_config('app.tenant_id', :tid, true)"),
                {"tid": str(tenant_id)},
            )
            yield session
```

18.3 Row-level security migration

migrations/0002_rls.sql

```
-- migrations/0002_rls.sql   (run by the `migrator` role only; app_rw never owns objects)
CREATE ROLE app_rw        LOGIN NOINHERIT NOBYPASSRLS;  -- API + workers
CREATE ROLE public_reader NOLOGIN;                      -- used by render_public only
GRANT public_reader TO app_rw;                          -- switched with SET LOCAL ROLE in txn

-- Fail-closed accessor: unset or empty setting -> NULL -> every policy matches zero rows.
CREATE OR REPLACE FUNCTION app_tenant() RETURNS uuid
LANGUAGE sql STABLE PARALLEL SAFE AS
$$ SELECT NULLIF(current_setting('app.tenant_id', true), '')::uuid $$;

CREATE TABLE account_runs (
  id               uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id        uuid NOT NULL REFERENCES tenants(id),
  account_id       uuid NOT NULL REFERENCES accounts(id),
  status           text NOT NULL DEFAULT 'QUEUED',
  graph_version    text NOT NULL,
  budget_json      jsonb NOT NULL,
  reserved_credits numeric(12,4) NOT NULL,
  idempotency_key  text NOT NULL,
  created_at       timestamptz NOT NULL DEFAULT now(),
  UNIQUE (tenant_id, idempotency_key)
);
ALTER TABLE account_runs ENABLE ROW LEVEL SECURITY;
ALTER TABLE account_runs FORCE  ROW LEVEL SECURITY;      -- applies even to the table owner
CREATE POLICY tenant_isolation ON account_runs
  USING      (tenant_id = (SELECT app_tenant()))          -- (SELECT ..) = evaluated once per query
  WITH CHECK (tenant_id = (SELECT app_tenant()));
GRANT SELECT, INSERT, UPDATE ON account_runs TO app_rw;

-- Two-output rule at the database layer: public_reader cannot see private rows at all.
CREATE POLICY public_only ON facts FOR SELECT TO public_reader
  USING (tenant_id = (SELECT app_tenant()) AND visibility = 'public');
GRANT SELECT ON facts TO public_reader;                   -- no grant at all on chunks

-- Library-owned LangGraph tables: thread_id = '<tenant_uuid>:<run_uuid>'
DO $$
DECLARE t text;
BEGIN
  FOREACH t IN ARRAY ARRAY['checkpoints','checkpoint_blobs','checkpoint_writes'] LOOP
    EXECUTE format('ALTER TABLE %I ENABLE ROW LEVEL SECURITY', t);
    EXECUTE format('ALTER TABLE %I FORCE ROW LEVEL SECURITY', t);
    EXECUTE format($p$CREATE POLICY tenant_prefix ON %I
      USING      (starts_with(thread_id, (SELECT app_tenant())::text || ':'))
      WITH CHECK (starts_with(thread_id, (SELECT app_tenant())::text || ':'))$p$, t);
    EXECUTE format('GRANT SELECT, INSERT, UPDATE, DELETE ON %I TO app_rw', t);
  END LOOP;
END $$;

-- Vector table: hash-partitioned by tenant, halfvec to halve memory, HNSW per partition.
CREATE EXTENSION IF NOT EXISTS vector;
CREATE TABLE chunks (
  id         uuid NOT NULL DEFAULT gen_random_uuid(),
  tenant_id  uuid NOT NULL,
  account_id uuid NOT NULL,
  doc_id     uuid NOT NULL,
  visibility text NOT NULL DEFAULT 'private' CHECK (visibility = 'private'),
  content    text NOT NULL,
  tsv        tsvector GENERATED ALWAYS AS (to_tsvector('english', content)) STORED,
  embedding  halfvec(1536) NOT NULL,
  PRIMARY KEY (tenant_id, id)
) PARTITION BY HASH (tenant_id);
-- generated in the migration: 16 partitions, each with
--   CREATE INDEX ON chunks_pN USING hnsw (embedding halfvec_cosine_ops) WITH (m=16, ef_construction=64);
ALTER TABLE chunks ENABLE ROW LEVEL SECURITY;
ALTER TABLE chunks FORCE  ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON chunks
  USING (tenant_id = (SELECT app_tenant())) WITH CHECK (tenant_id = (SELECT app_tenant()));
GRANT SELECT, INSERT, DELETE ON chunks TO app_rw;
```

18.4 Research graph and tool ledger

src/agents/graph.py

```
"""src/agents/graph.py - research graph (reduced to the load-bearing structure)."""
from __future__ import annotations

import hashlib
import json
import operator
from typing import Annotated, Any, Awaitable, Callable, TypedDict
from uuid import UUID

from langgraph.graph import END, START, StateGraph
from langgraph.types import RetryPolicy, Send
from sqlalchemy import text

from src.db.session import tenant_txn

GRAPH_VERSION = "2026-10-01.1"          # pinned on account_runs.graph_version at creation
BRANCHES = ["news", "jobs", "people", "tech"]


class RunState(TypedDict, total=False):
    schema_version: int
    tenant_id: str
    run_id: str
    domain: str
    budget_usd: float
    spent_usd: float
    facts: Annotated[list[dict[str, Any]], operator.add]   # reducer: branches append in parallel
    status: str


class BudgetExceeded(Exception):
    pass


async def ledgered_call(tenant_id: str, run_id: str, tool: str, args: dict[str, Any],
                        fn: Callable[..., Awaitable[dict[str, Any]]]) -> dict[str, Any]:
    """Exactly-once *billing* for paid calls: replay from the ledger if this call already ran."""
    h = hashlib.sha256(json.dumps([tool, args], sort_keys=True, separators=(",", ":")).encode()).hexdigest()
    async with tenant_txn(UUID(tenant_id)) as s:
        row = (await s.execute(
            text("SELECT response FROM tool_calls WHERE run_id = :r AND args_hash = :h"),
            {"r": run_id, "h": h})).first()
        if row:
            return row.response
    result = await fn(**args)                       # the paid call (broker or LLM)
    async with tenant_txn(UUID(tenant_id)) as s:
        await s.execute(text(
            "INSERT INTO tool_calls (tenant_id, run_id, tool, args_hash, response, cost_usd) "
            "VALUES ((SELECT app_tenant()), :r, :t, :h, CAST(:resp AS jsonb), :c) "
            "ON CONFLICT (run_id, args_hash) DO NOTHING"),
            {"r": run_id, "t": tool, "h": h, "resp": json.dumps(result), "c": result.get("cost_usd", 0)})
    return result


async def baseline(state: RunState) -> dict[str, Any]:
    return {"schema_version": 1, "spent_usd": 0.0, "facts": []}


def fan_out(state: RunState) -> list[Send]:
    if state.get("spent_usd", 0.0) >= state["budget_usd"]:
        raise BudgetExceeded(state["run_id"])        # runner maps this to BUDGET_EXCEEDED
    return [Send("fetch_branch", {**state, "branch": b}) for b in BRANCHES]


async def fetch_branch(state: dict[str, Any]) -> dict[str, Any]:
    async def broker(branch: str, domain: str) -> dict[str, Any]:
        raise NotImplementedError("call the allow-listed broker adapter here")
    out = await ledgered_call(state["tenant_id"], state["run_id"], f"broker:{state['branch']}",
                              {"branch": state["branch"], "domain": state["domain"]}, broker)
    # untrusted text goes to the tool-less extractor tier, never to the planner
    return {"facts": out.get("facts", []), "spent_usd": out.get("cost_usd", 0.0)}


async def distill(state: RunState) -> dict[str, Any]:
    return {"status": "COMPLETED"}


def build_graph() -> StateGraph:
    g = StateGraph(RunState)
    g.add_node("baseline", baseline)
    g.add_node("fetch_branch", fetch_branch,
```

```
                retry_policy=RetryPolicy(max_attempts=3, initial_interval=2.0, backoff_factor=2.0))
    g.add_node("distill", distill)
    g.add_edge(START, "baseline")
    g.add_conditional_edges("baseline", fan_out, ["fetch_branch"])
    g.add_edge("fetch_branch", "distill")
    g.add_edge("distill", END)
    return g
```

18.5 Time-sliced resumable runner

src/agents/runner.py

```
"""src/agents/runner.py - time-sliced, resumable execution. The checkpoint is the source of truth."""
from __future__ import annotations

import asyncio
import os
from typing import Any
from uuid import UUID

import psycopg
from langgraph.checkpoint.postgres.aio import AsyncPostgresSaver
from psycopg.rows import dict_row

from src.agents.graph import BudgetExceeded, build_graph

SLICE_SECONDS = int(os.getenv("SLICE_SECONDS", "240"))


async def _dedicated_tenant_conn(tenant_id: UUID) -> psycopg.AsyncConnection:
    conn = await psycopg.AsyncConnection.connect(
        os.environ["CHECKPOINT_DATABASE_URL"],      # app_rw, direct to writer (no txn-mode proxy)
        autocommit=True, row_factory=dict_row, prepare_threshold=0)
    # Session-scoped (is_local=false) because autocommit has no long transaction. Safe only because
    # this connection is private to one run and is closed in `finally`; it never returns to a pool.
    await conn.execute("SELECT set_config('app.tenant_id', %s, false)", (str(tenant_id),))
    return conn


async def execute_slice(tenant_id: UUID, run_id: UUID, domain: str, budget_usd: float) -> dict[str, Any]:
    config = {"configurable": {"thread_id": f"{tenant_id}:{run_id}"}, "recursion_limit": 60}
    conn = await _dedicated_tenant_conn(tenant_id)
    try:
        # saver.setup() is run once by the migrator job, never here.
        graph = build_graph().compile(checkpointer=AsyncPostgresSaver(conn))
        snap = await graph.aget_state(config)
        if snap.values and not snap.next:
            return {"done": True, "status": snap.values.get("status")}   # already finished
        initial = None if snap.values else {
            "tenant_id": str(tenant_id), "run_id": str(run_id), "domain": domain,
            "budget_usd": budget_usd}
        try:
            async with asyncio.timeout(SLICE_SECONDS):
                await graph.ainvoke(initial, config)       # None => resume from last checkpoint
        except TimeoutError:
            return {"done": False}                          # next slice resumes; ledger prevents re-billing
        except BudgetExceeded:
            return {"done": True, "status": "BUDGET_EXCEEDED"}
        final = await graph.aget_state(config)
        if final.next:                                      # paused on an interrupt (HITL gate)
            return {"done": False, "awaiting": True}
        return {"done": True, "status": final.values.get("status", "COMPLETED")}
    finally:
        await conn.close()
```

18.6 Orchestration loop

src/orchestrator/functions.py

```
"""src/orchestrator/functions.py - Inngest is only the outer loop: trigger, slice, wait, notify."""
from __future__ import annotations

import datetime
import os
from uuid import UUID

import inngest

from src.agents.runner import execute_slice

client = inngest.Inngest(app_id="ai-saas", is_production=os.getenv("ENV") == "prod")
MAX_SLICES = int(os.getenv("MAX_SLICES", "60"))        # 60 x 240 s = 4 h hard ceiling


@client.create_function(
    fn_id="run-research",
    trigger=inngest.TriggerEvent(event="research/run.requested"),
    retries=2,                                          # infrastructure failures only; node retries live in LangGraph
    concurrency=[inngest.Concurrency(limit=int(os.getenv("TENANT_CONCURRENCY", "5")),
                                     key="event.data.tenant_id")],   # per-tenant fairness
)
async def run_research(ctx: inngest.Context, step: inngest.Step) -> dict:
    d = ctx.event.data
    tenant_id, run_id = UUID(d["tenant_id"]), UUID(d["run_id"])
    out: dict = {"done": False}
    for i in range(MAX_SLICES):
        out = await step.run(
            f"slice-{i}",
            lambda: execute_slice(tenant_id, run_id, d["domain"], float(d["budget_usd"])))
        if out.get("done"):
            break
        if out.get("awaiting"):
            await step.wait_for_event(
                f"approval-{i}", event="research/run.approved",
                if_exp="event.data.run_id == async.data.run_id",
                timeout=datetime.timedelta(hours=24))
    else:
        out = {"done": True, "status": "FAILED"}          # ceiling reached: settle + refund in finalizer
    await step.send_event("emit-completed", inngest.Event(
        name="research/run.completed",
        data={"tenant_id": d["tenant_id"], "run_id": d["run_id"], "status": out.get("status")}))
    return out
```

18.7 Authentication

src/api/auth.py

```
"""src/api/auth.py - verify JWT, derive identity. Tenant comes from the token, never from the body."""
from __future__ import annotations

import os
from dataclasses import dataclass
from uuid import UUID

import jwt
from fastapi import Depends, Header, HTTPException
from jwt import PyJWKClient

_jwks = PyJWKClient(os.environ["JWKS_URL"], cache_keys=True)


@dataclass(frozen=True)
class Identity:
    user_id: str
    tenant_id: UUID
    role: str


async def identity(authorization: str = Header(...)) -> Identity:
    token = authorization.removeprefix("Bearer ").strip()
    try:
        key = _jwks.get_signing_key_from_jwt(token).key
        claims = jwt.decode(token, key, algorithms=["RS256"],
                            audience=os.environ["JWT_AUDIENCE"], issuer=os.environ["JWT_ISSUER"],
                            options={"require": ["exp", "iat", "sub"]})
        return Identity(claims["sub"], UUID(claims["tenant_id"]), claims["role"])
    except (jwt.PyJWTError, KeyError, ValueError):
        raise HTTPException(status_code=401, detail="invalid token")


def require_role(*roles: str):
    async def dep(ident: Identity = Depends(identity)) -> Identity:
        if ident.role not in roles:        # also re-check membership in DB for revocation (cached 60 s)
            raise HTTPException(status_code=403, detail="forbidden")
        return ident
    return dep
```

18.8 Run creation: idempotent, reserved, outbox-safe

src/api/runs.py

```
"""src/api/runs.py - create a run: idempotent, credit-reserving, outbox-safe."""
from __future__ import annotations

import logging
from typing import Literal
from uuid import UUID, uuid4

import inngest
from fastapi import APIRouter, Depends, Header, HTTPException
from pydantic import BaseModel, Field
from sqlalchemy import text

from src.agents.graph import GRAPH_VERSION
from src.api.auth import Identity, require_role
from src.db.session import tenant_txn
from src.orchestrator.functions import client as inngest_client

log = logging.getLogger(__name__)
router = APIRouter(prefix="/v1")
COST = {"standard": 6.0, "deep": 18.0}                    # credits reserved up front (calibrate with data)


class RunRequest(BaseModel):
    domain: str = Field(pattern=r"^[a-z0-9]([a-z0-9-]*[a-z0-9])?(\.[a-z0-9-]+)+$", max_length=253)
    depth: Literal["standard", "deep"] = "standard"


@router.post("/runs", status_code=202)
async def create_run(body: RunRequest, idem: str = Header(alias="Idempotency-Key", min_length=8, max_length=64),
                     ident: Identity = Depends(require_role("admin", "seller"))) -> dict:
    est = COST[body.depth]
    run_id = uuid4()
    async with tenant_txn(ident.tenant_id) as s:
        claimed = (await s.execute(text(
            "WITH a AS ("
            "  INSERT INTO accounts (tenant_id, domain) VALUES ((SELECT app_tenant()), :d) "
            "  ON CONFLICT (tenant_id, domain) DO UPDATE SET domain = EXCLUDED.domain RETURNING id) "
            "INSERT INTO account_runs (id, tenant_id, account_id, graph_version, budget_json, "
            "reserved_credits, idempotency_key) "
            "SELECT :id, (SELECT app_tenant()), a.id, :gv, CAST(:b AS jsonb), :est, :k FROM a "
            "ON CONFLICT (tenant_id, idempotency_key) DO NOTHING RETURNING id"),
            {"id": run_id, "gv": GRAPH_VERSION, "b": '{"usd": %s}' % est, "est": est,
             "k": idem, "d": body.domain})).first()
        if claimed is None:                                  # replay of an earlier request
            prev = (await s.execute(text(
                "SELECT id, status FROM account_runs WHERE idempotency_key = :k"), {"k": idem})).one()
            return {"run_id": str(prev.id), "status": prev.status}
        ok = (await s.execute(text(                          # atomic: only succeeds if balance suffices
            "UPDATE wallets SET balance = balance - :e WHERE tenant_id = (SELECT app_tenant()) "
            "AND balance >= :e RETURNING balance"), {"e": est})).first()
        if ok is None:
            raise HTTPException(402, "insufficient credits")   # raising rolls the whole txn back
        await s.execute(text(
            "INSERT INTO wallet_ledger (tenant_id, run_id, entry, amount, balance_after) "
            "VALUES ((SELECT app_tenant()), :r, 'reserve', :e, :bal)"),
            {"r": run_id, "e": -est, "bal": ok.balance})
    try:                                                     # after commit; reconciler re-sends if this fails
        await inngest_client.send(inngest.Event(
            id=str(run_id), name="research/run.requested",   # event id = idempotent delivery
            data={"tenant_id": str(ident.tenant_id), "run_id": str(run_id),
                  "domain": body.domain, "budget_usd": est}))
    except Exception:
        log.warning("event send failed; reconciler will retry", extra={"run_id": str(run_id)})
    return {"run_id": str(run_id), "status": "QUEUED"}
```

If the event send fails after commit, the run remains QUEUED. The reconciler (a singleton job) lists stuck runs through a narrow definer function that returns only tenant and run ids, then re-sends each event inside that tenant's context. Because the event id equals the run id, duplicate delivery is harmless.

18.9 Isolation test suite (CI hard gate)

tests/isolation/test_rls.py

```
"""tests/isolation/test_rls.py - the hard CI gate. Fixtures create tenants A and B as the migrator."""
from uuid import UUID

import psycopg
import pytest
from sqlalchemy import text

from src.db.session import SessionLocal, tenant_txn

pytestmark = pytest.mark.asyncio


async def test_cross_tenant_read_returns_nothing(tenant_a: UUID, run_of_b: UUID):
    async with tenant_txn(tenant_a) as s:
        rows = (await s.execute(text("SELECT 1 FROM account_runs WHERE id = :i"), {"i": run_of_b})).all()
    assert rows == []


async def test_unset_tenant_fails_closed():
    async with SessionLocal() as s:                        # no set_config at all
        n = (await s.execute(text("SELECT count(*) FROM account_runs"))).scalar_one()
    assert n == 0


async def test_cross_tenant_write_is_rejected(tenant_a: UUID, tenant_b: UUID, account_of_b: UUID):
    with pytest.raises(Exception, match="row-level security"):
        async with tenant_txn(tenant_a) as s:
            await s.execute(text(
                "INSERT INTO account_runs (tenant_id, account_id, graph_version, budget_json, "
                "reserved_credits, idempotency_key) VALUES (:t, :a, 'x', '{}', 1, 'k-12345678')"),
                {"t": tenant_b, "a": account_of_b})


async def test_context_does_not_leak_through_pool(tenant_a: UUID):
    async with tenant_txn(tenant_a):
        pass
    async with SessionLocal() as s:                        # very likely the same pooled connection
        v = (await s.execute(text("SELECT current_setting('app.tenant_id', true)"))).scalar_one()
    assert v in (None, "")


async def test_checkpoint_prefix_policy(tenant_a_dsn: str, tenant_b: UUID):
    async with await psycopg.AsyncConnection.connect(tenant_a_dsn, autocommit=True) as c:
        with pytest.raises(psycopg.errors.InsufficientPrivilege):
            await c.execute(
                "INSERT INTO checkpoints (thread_id, checkpoint_ns, checkpoint_id, checkpoint, metadata) "
                "VALUES (%s, '', 'c1', '{}', '{}')", (f"{tenant_b}:some-run",))
```

18.10 Container, local environment and CI

Dockerfile

```
# syntax=docker/dockerfile:1
FROM python:3.12-slim AS build
ENV UV_COMPILE_BYTECODE=1 UV_LINK_MODE=copy PIP_NO_CACHE_DIR=1
RUN pip install uv
WORKDIR /app
COPY pyproject.toml uv.lock ./
RUN uv sync --frozen --no-dev --no-install-project
COPY src ./src

FROM python:3.12-slim
RUN useradd --system --uid 10001 --no-create-home app
WORKDIR /app
COPY --from=build /app /app
ENV PATH="/app/.venv/bin:$PATH" PYTHONUNBUFFERED=1
USER 10001
EXPOSE 8080
CMD ["uvicorn", "src.api.main:app", "--host", "0.0.0.0", "--port", "8080", "--workers", "2"]
```

compose.yml

```
services:
  db:
    image: pgvector/pgvector:pg16
    environment: { POSTGRES_PASSWORD: dev, POSTGRES_DB: ai }
    ports: ["5432:5432"]
    healthcheck: { test: ["CMD-SHELL", "pg_isready -U postgres"], interval: 5s, retries: 10 }
  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]
  api:
    build: .
    depends_on: { db: { condition: service_healthy }, redis: { condition: service_started } }
    environment:
      DATABASE_URL: postgresql+asyncpg://app_rw:dev@db/ai
      CHECKPOINT_DATABASE_URL: postgresql://app_rw:dev@db/ai
      REDIS_URL: redis://redis:6379/0
      ENV: dev
    ports: ["8080:8080"]
  inngest:
    image: inngest/inngest
    command: inngest dev -u http://api:8080/api/inngest
    ports: ["8288:8288"]
```

.github/workflows/ci.yml

```
name: ci
on: { pull_request: {}, push: { branches: [main] } }
jobs:
  verify:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: pgvector/pgvector:pg16
        env: { POSTGRES_PASSWORD: dev }
        ports: ["5432:5432"]
        options: >-
          --health-cmd "pg_isready -U postgres" --health-interval 5s --health-retries 10
    steps:
      - uses: actions/checkout@v4            # pin third-party actions by commit SHA in production
      - uses: astral-sh/setup-uv@v5
      - run: uv sync --frozen
      - run: uv run ruff check . && uv run mypy src
      - run: uv run alembic upgrade head      # applies RLS migrations as the migrator role
      - run: uv run pytest tests/isolation -x # HARD GATE: any cross-tenant read/write fails the build
      - run: uv run pytest -m "not eval" --cov=src --cov-fail-under=80
      - run: docker build -t ai-saas:${{ github.sha }} .
      - run: trivy image --exit-code 1 --severity HIGH,CRITICAL ai-saas:${{ github.sha }}
  eval-gate:
    if: github.ref == 'refs/heads/main'
    needs: verify
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: uv run pytest -m eval --eval-baseline=main   # blocks promotion on regression
```

19. Risks, Open Questions and Prototype Plan

19.1 Risk register

Risk	Likelihood	Impact	Mitigation	Owner
Cross-tenant leakage through RLS misconfiguration	Low	Severe	Layered isolation, migration lint, isolation gate, planted canaries, external test	Architect
Broker cost exceeds price per run	High	High	Per-run caps, probe then escalate, ledger-driven negotiation, depth-based pricing	Backend lead
Silent prompt-cache invalidation	Medium	Medium	Deterministic serialisation, cache-ratio SLO, canary build	AI engineer
Indirect prompt injection from scraped or private content	High	High	Quarantine extractor, typed tools, adversarial corpus	AI engineer
Workflow engine limits or outage	Medium	Medium	Slice design, reconciler, documented Temporal alternative	Architect
Filtered ANN under-recall for small tenants	Medium	Medium	Partitions, iterative scans, per-tenant recall probes	AI engineer
Checkpoint and ledger growth	Medium	Low	Seven-day post-terminal retention, offload large payloads to S3	Backend lead
Privacy or licence challenge on people data and scraped sources	Medium	Severe	Legal review before launch, source licences, objection handling	Founder + counsel
Golden set decay misleads evals	High	Medium	Capture dates, quarterly re-verification, paired comparisons	AI engineer
Regional failover does not meet RTO	Medium	High	Scripted runbook, quarterly game-day, dependency checks	Platform

19.2 Build this first (Phase 0 spikes with pass criteria)

#	Spike	Pass criterion	If it fails
1	Isolation: forced RLS, non-owner role, pooled connections, checkpoint prefix policy, partitioned vectors with canary vectors	100 concurrent mixed-tenant requests over a 5-connection pool produce zero cross-tenant rows; unset context returns zero rows	Revisit tenancy tier defaults before any feature work
2	Durability: two-node graph, kill the process mid-fan-out repeatedly, resume via slices	Run completes after 5 random kills; ledger shows each paid call exactly once	Evaluate Temporal for the outer loop
3	Prompt caching: run the baseline prompt 10 times in a loop with realistic tools	Cache-read ratio at least 70% from the second call; no invalidation across slices of 6 minutes	Adopt the longer cache lifetime or restructure the prefix
4	Broker economics: run 20 real accounts at standard depth	Cost per run within the planned cap; coverage of expected attributes at least 80%	Reprice or narrow the broker set
5	Autoscaling: burst of 50 simultaneous runs against stub brokers	Pods scheduled within a minute; consolidation after the burst; no run lost on spot reclamation	Adjust pool sizing and disruption budgets

19.3 Open questions for the product owner

- What is the list price per run or credit, and which depth tiers will exist? Section 16 shows the break-even by tier.
- Which regions must data stay in for the first ten customers, and do any require a silo tier on day one?
- Which people-data attributes are in scope for the account brief, given the privacy review?
- Is the operator console a long-term product surface or a transitional tool?
- Which CRM and storage connectors does the first Phase B customer actually need?

Appendix A. Glossary

Term	Meaning
RLS	PostgreSQL row-level security: per-row policies evaluated by the database
Checkpoint	Serialised graph state saved after each superstep, enabling resume
Slice	A bounded execution window (default 240 s) after which the run yields and resumes
Ledger (tool)	Table of paid calls keyed by run and argument hash, used for replay and attribution
Ledger (wallet)	Insert-only record of credit reservations, commits, refunds and top-ups
Lineage group	Set of sources sharing a common upstream origin, counted as one for corroboration
Quarantine extractor	Tool-less model step that turns untrusted text into schema-validated JSON
Two-output rule	Client-facing deliverables use public data only; internal views may blend private data
halfvec	pgvector 16-bit floating point vector type
HNSW	Hierarchical navigable small world approximate nearest-neighbour index
RPO / RTO	Recovery point objective (data loss window) and recovery time objective
Probe	Cheap partial refresh that decides whether a full run is needed

Appendix B. Verification Register

Items I could not confirm from first-party sources while preparing this document. Resolve each before the dependent decision is locked. Owner and date columns are left blank for the team to complete.

#	Item to verify	Why it matters	Chapter	Owner / date
V1	Current Anthropic model IDs, prices, cache lifetimes, price multipliers, minimum cacheable length, breakpoint limit	Cost model and cache design	10, 16	
V2	Aurora Global Database support for the chosen engine version in ap-south-2; measured inter-region latency	DR plan and RPO	11	
V3	pgvector version on Aurora; availability of iterative index scans and halfvec limits	Filtered retrieval design	7	
V4	Inngest step time and payload limits, concurrency-key syntax, failover and data-residency behaviour; Python SDK call signatures	Slice design and starter code	5, 18	
V5	Zero-retention and no-training terms for LLM, embedding and speech vendors	Phase B privacy promise	10, 14	
V6	Speech recognition pricing and diarization quality on real call audio	Phase B economics	8, 16	
V7	Broker contract pricing, rate limits, and permitted use of derived data	Unit economics and compliance	14, 16	
V8	Karpenter and spot savings on our workload	Compute cost	11	
V9	OWASP LLM Top 10 edition in force at adoption	Security mapping	14	
V10	Applicability of DPDP Act, GDPR and similar rules to the people-data in briefs	Launch readiness	14	
V11	LangGraph checkpoint library version: setup() behaviour, connection requirements, adelete_thread	Durability implementation	5, 18	

Document history: v1 (single-pass architecture draft) reviewed against first principles; v2.0 (this document) corrects 19 issues, adds the isolation chapter, grading model, slice-based durability, billing ledger, Phase 0 plan and a parametrised cost model.